UNIVERSIDADE FEDERAL DO PIAUÍ – UFPI
CENTRO DE TECNOLOGIA – CT
CAMPUS MINISTRO PETRÔNIO PORTELLA
BACHARELADO EM ENGENHARIA MECÂNICA

DÁRIO CARVALHO VIVEIROS

ANALYSIS OF PASSIVE VIBRATION MITIGATION AND BAND GAP LOCALIZATION IN TLP TENDONS USING A PERIODICALLY SEGMENTED EULER-BERNOULLI BEAM MODEL

TERESINA – PI
2025

DÁRIO CARVALHO VIVEIROS

ANALYSIS OF PASSIVE VIBRATION MITIGATION AND BAND GAP LOCALIZATION IN TLP TENDONS USING A PERIODICALLY SEGMENTED EULER-BERNOULLI BEAM MODEL

This work was presented to the Center of Technology (CT) of the Federal University of Piauí (UFPI), as a requirement for obtaining the degree of Bachelor in Mechanical Engineering.

Advisor: Simone dos Santos Hoefel, Ph.D.

TERESINA – PI
2025

FICHA CATALOGRÁFICA
Universidade Federal do Piauí
Biblioteca Comunitária Jornalista Carlos Castello Branco
Divisão de Representação da Informação

V857a Viveiros, Dário Carvalho.
Analysis of passive vibration mitigation and band gap localization in tlp tendons using a periodically segmented euler-bernoulli beam model / Dário Carvalho Viveiros. – 2025.
99 f.

Trabalho de Conclusão de Curso (Graduação) – Universidade Federal do Piauí, Centro de Tecnologia, Bacharelado em Engenharia Mecânica, Teresina, 2025.
"Orientador(a): Simone dos Santos Hoefel, Ph.D.".

1. Plataformas de Pernas Tracionadas. 2. Vigas Periódicas. 3. Mitigação de Vibrações. 4. Faixas de Atenuação. 5. Método dos Elementos Finitos. I. Hoefel, Simone dos Santos. II. Título.

CDD 621

Bibliotecária: Francisca das Chagas Dias Leite – CRB3/1004

DÁRIO CARVALHO VIVEIROS

ANALYSIS OF PASSIVE VIBRATION MITIGATION AND BAND GAP LOCALIZATION IN TLP TENDONS USING A PERIODICALLY SEGMENTED EULER-BERNOULLI BEAM MODEL

This work was presented to the Center of Technology (CT) of the Federal University of Piauí (UFPI), as a requirement for obtaining the degree of Bachelor in Mechanical Engineering.

Advisor: Simone dos Santos Hoefel, Ph.D.

Approved on 10 / December / 2025.

EXAMINATION COMMITTEE

Documento assinado digitalmente
SIMONE DOS SANTOS HOEFEL
Data: 19/12/2025 19:55:52-0300
Verifique em https://validar.iti.gov.br

Prof. Simone dos Santos Hoefel, Ph.D.
Universidade Federal do Piauí

Documento assinado digitalmente
ANDERSON SOARES DA COSTA AZEVEDO
Data: 19/12/2025 20:24:51-0300
Verifique em https://validar.iti.gov.br

Anderson Soares da Costa Azevêdo, Ph.D.
Universidade de São Paulo

José Pereira Ramos Junior

José Pereira Ramos Junior, M.Sc.
Universidade de São Paulo

To Malu, Simone, Raissa, Iasmin,
and Glória.

Thank you for being by my side so often,
and helping me through even the hardest
of times. Each of you changed me
for the better, in one way or another.
For that I'll be forever grateful.

ACKNOWLEDGMENT

Above all I thank Professor Simone dos Santos Hoefel, for all the time and effort spent in my orientation through the years. I would not have come this far without you. I extend my thanks to all my colleagues from the Laboratório de Métodos de Modelagem Computacional (LAMEC). Special thanks go to my good friend, Thierry Miqueias Melo Madeira, for extensive help in achieving the results in this work. And to Davi Kauê De Sousa Gomes, for extended work on our common LaTeX 2_ε environment, despite his severe research routine. Lastly, I thank my parents for overall support during the period of this research.

‘‘I’m a leaf on the wind,

watch how I soar.’’

Hoban Washburne

RESUMO

As cargas dinâmicas induzidas por ondas em Plataformas de Pernas Tracionadas representam uma ameaça à integridade estrutural, sobretudo pela transmissão de vibrações ao longo dos tendões. Neste contexto, este estudo investiga o uso de configurações periódicas nesses elementos como estratégia passiva para mitigação de vibrações. A partir da teoria de vigas de Euler-Bernoulli, desenvolveu-se uma rotina numérica baseada no Método dos Elementos Finitos, implementada em MATLAB e validada com a teoria analítica, para analisar a formação de faixas de atenuação de frequência, onde a propagação de ondas é suprimida. Duas configurações periódicas são avaliadas: vigas escalonadas (periodicidade geométrica) e vigas bi-materiais (periodicidade de material). Uma análise da formação de zonas de atenuação é realizada por meio de um parâmetro de saturação periódica. Os resultados mostram que, embora ambas as configurações sejam capazes de gerar band gaps, as vigas escalonadas apresentam maior capacidade de sintonização e produzem bandas de atenuação mais amplas. Assim, a otimização da periodicidade geométrica destaca-se como uma abordagem de projeto robusta para a filtragem de frequências de vibração críticas em sistemas marítimos.

Palavras-chave: Plataformas de Pernas Tracionadas, Vigas Periódicas, Mitigação de Vibrações, Faixas de Atenuação, Método dos Elementos Finitos.

ABSTRACT

Wave-induced dynamic loads on Tension Leg Platforms pose a threat to structural integrity, particularly due to the transmission of vibrations along the tendons. In this context, this study investigates the use of periodic configurations in these elements as a passive strategy for vibration mitigation. Based on Euler-Bernoulli beam theory, a numerical routine based on the Finite Element Method was developed and implemented in MATLAB, and validated against analytical formulation, to analyze the formation of band gaps, where wave propagation is suppressed. Two periodic configurations are evaluated: stepped beams (geometric periodicity) and bi-material beams (material periodicity). An analysis of band gaps formation is achieved via a periodic saturation parameter. The results demonstrate that, although both configurations are capable of generating band gaps, stepped beams exhibit higher tunability and yield broader attenuation bands. Thus, the optimization of geometric periodicity stands out as a robust design approach for filtering critical vibration frequencies in offshore systems.

Keywords: Tension Leg Platforms, Periodic Beams, Vibration Mitigation, Bang Gaps, Finite Element Method.

LIST OF FIGURES

Figure 1 - Example of Tension Leg Platform. 13

Figure 2 - Beam with n step changes in cross sectional area..... 20

Figure 3 - Compatibility of internal forces around steps. 21

Figure 4 - Three-segment unit cell for (a) stepped and (b) bi-material beam configurations, with total length l. 21

Figure 5 - Euler-Bernoulli beam element with nodal degrees of freedom: deflection (V^e) and slope (θ^e). 22

Figure 6 - Unit cell meshing for (a) stepped and (b) bi-material configuration. 24

Figure 7 - Beam with one step change in cross sectional area. 26

Figure 8 - Natural modes number (a) one and (b) two: Frequency ratios of a stepped circular rod for various boundary conditions. 27

Figure 9 - Third mode: Frequency ratios of a stepped circular rod for various boundary conditions. 27

Figure 10 - Band gaps for (a) bimaterial and (b) stepped beam configuration, with $\alpha = 1/2$. 29

Figure 11 - Frequency gaps broadening by saturation parameter α for (a) bi-material and (b) stepped beams, for natural modes 7 through 11..... 30

Figure 12 - Band gap absence for bi-material beam configuration on points (a) $\alpha = 3/16$ and (b) $\alpha = 13/16$. 30

Figure 13 - Band gaps for stepped beam configuration for (a) $\alpha = 6/16$ and (b) $\alpha = 14/16$. 31

Figure 14 - Band gap broadening by saturation parameter for (a) bi-material and (b) stepped beams, for natural modes 18 through 22. 31

LIST OF TABLES

Table 1 - Fundamental frequencies for three types of segmented beams..... 26

Table 2 - Diameter ratios relation to natural frequencies for various boundary conditions... 26

Table 3 - Stepped and bimaterial configurations for Tension Leg Platform..... 28

Table 4 - First 10 natural frequencies (in rad/s) for stepped and bi-material configurations, with $\alpha = 1/2$. 29

LIST OF SYMBOLS

Symbol	Description	Order	Size	Units
A	Cross-sectional area	Scalar	-	m^2
A_i	Mode shape sine amplitude constant	Scalar	-	-
a	Reference length of Euler-Bernoulli beam element	Scalar	-	-
b	Beam element width	Scalar	-	m
B_i	Mode shape cosine amplitude constant	Scalar	-	-
C_i	Mode shape hyperbolic sine amplitude constant	Scalar	-	-
c_i	Beam deflection approximation function constants	Scalar	-	-
D_i	Mode shape hyperbolic cosine amplitude constant	Scalar	-	-
d	Cross sectional diameter	Scalar	-	m
E	Young Modulus	Scalar	-	Pa
h	Beam element height	Scalar	-	m
I	Moment of Inertia	Scalar	-	m^4
$\mathbf{[K]}$	Global stiffness matrix	Matrix	$4n \times 4n$	-
$\{\mathbf{k}\}_e$	Elemental stiffness matrix	Matrix	4×4	-
k_R	Torsional spring constant	Scalar	-	$N \cdot \frac{m}{rad}$
k_T	Linear spring constant	Scalar	-	N/m
L	Total beam length	Scalar	-	m
l	Unit cell length	Scalar	-	m
l_e	Beam element length	Scalar	-	m
M_i	Inner Moment of beam element	Scalar	-	$N \cdot m$
$\mathbf{[M]}$	Global mass matrix	Matrix	$4n \times 4n$	-
$\{\mathbf{m}\}_e$	Elemental mass matrix	Matrix	4×4	-
$\mathbf{[N]}$	Shape functions	Vector	1×4	-
n	Total number of beam segments or elements	Scalar	-	-
n_c	Number of unit cells	Scalar	-	-
Q_i	Sheer Force of beam element	Scalar	-	N
t	Time	Scalar	-	s
V	Beam element deflection	Scalar	-	m
V^e	Nodal deflection	Scalar	-	m

Symbol	Description	Order	Size	Units
$\tilde{V}^e$	Elemental beam deflection approximation function	Scalar	-	m
$\{\mathbf{v}\}$	Nodal displacement vector	Vector	$n \times 1$	m
$\{\mathbf{v}\}_e$	Elemental nodal displacement vector	Vector	4×1	m
W_i	Gauss-Legendre integration weight functions	Scalar	-	-
w	Beam mode shape	Scalar	-	m
w_i	Beam segment deflection	Scalar	-	m
x	Beam longitudinal coordinate	Scalar	-	m
x_i	Beam segment or elemental coordinate	Scalar	-	m
α	Saturation parameter	Scalar	-	-
β	Frequency parameter	Scalar	-	m^{-4}
$\Delta\omega$	Frequency Gap or Band Gap	Scalar	-	rad/s
θ^e	Nodal rotation	Scalar	-	rad
λ	Beam natural frequency	Scalar	-	rad/s
ρ	Volumetric density	Scalar	-	kg/m^3
ω	Beam natural frequency	Scalar	-	rad/s

CONTENTS

1. INTRODUCTION 13

1.1. Contextualization And Motivation 13

1.2. Work Details 14

1.3. Objectives 14

2. THEORETICAL BACKGROUND 15

2.1. Historic Prospects Of TLP Tendon Modeling 15

2.2. Application Of Metamaterials For Vibration Mitigation In TLPs 16

3. METHODOLOGY 19

3.1. Analytical Stepped Beam Formulation 19

3.2. Modeling Periodic Structures via Stepped Beam Theory 21

3.3. Modeling Periodic Structures Via FEM 22

4. RESULTS 25

4.1. Validations of Analytical Routines 25

4.2. Passive Mitigation Analysis 28

5. CONCLUSIONS 32

REFERENCES 34

APPENDIX A – ANALYTICAL MATLAB ROUTINES 37

APPENDIX B – FEM MATLAB ROUTINES 51

1 INTRODUCTION

1.1 Contextualization And Motivation

Offshore platforms operate in harsh marine environments, continuously exposed to wave action, which induces dynamic loads on their structural components – particularly on the support legs and tendons of Tension Leg Platforms (as seen on Figure 1). These loads propagate as mechanical vibrations throughout the structure, potentially compromising its integrity, operational performance, and service life (Chakrabarti, 2005).

Figure 1 – Example of Tension Leg Platform.

Fonte: Petrobrás.

To ensure the reliability and longevity of such systems, vibration mitigation has become a key research focus. Among passive control strategies, one finds the extensive implementation of tuned vibration absorbers, which have been rigorously investigated for deep-water applications. Considerable research efforts have demonstrated the efficacy of auxiliary damping devices –ranging from vertical caissons and Tuned Liquid Column Dampers (TLCD) to multiple Tuned Mass Damper (TMD) configurations, in suppressing resonant heave and pitch motions under stochastic wave excitation (Kandasamy *et al.*, 2016).

An alternative mean for passive mitigation lies in the introduction of intrinsic structural modifications. This perspective motivates the investigation of periodic structures, also called phononic or elastic metamaterials, where the periodic modulation of the tendon's geometry or material distribution acts as a passive filter for wave propagation. By inducing attenuation bands, these systems eliminate the need for complex auxiliary mechanisms. Together, these characteristics make this approach particularly advantageous for ensuring long-term robustness of TLP tendons.

Consequently, understanding the underlying effects and optimizing these periodic configurations can lead to increased tunability, *i.e.*, control of the bandwidth and location of attenuation bands. This leads to improved vibration control in such offshore platforms, allowing for safer, more durable, and sustainable marine operations.

1.2 Work Details

Section 1 presents context and motivation for the development of the current study. Section 2 shows a brief history of TLP tendon modeling focused on the use of various beam theories. Then, section 3 presents the EBT model developed via analytical and FEM formulations. While section 4 presents validations of relevant routines, and the simulation of a tendon-like structure. Finally, section 5 displays the main conclusions drawn from this research.

1.3 Objectives

The primary objective of this research is to analyze vibration mitigation on tendon-like structures for TLPs. To this end, the study employs periodic Euler-Bernoulli beams, configured as both stepped and bi-material structures, as simplified representations to investigate the effectiveness of periodicity as a passive strategy for reducing mechanical vibrations. The following objectives work as a basis towards that goal:

- Develop an analytical model for the dynamic response of periodic beams based on the Stepped Euler-Bernoulli Beam Theory (EBT).
- Implement a numerical model using the Finite Element Method (FEM) in a MATLAB® environment to solve the generalized eigenvalue problem associated with the proposed configurations.
- Analyze the formation and location of frequency band gaps in both stepped (geometric periodicity) and bimaterial (material periodicity) beam configurations.
- Evaluate the influence of unit cell configuration on the natural frequencies, and resulting band gaps generated for the structure.

2 THEORETICAL BACKGROUND

2.1 Historic Prospects Of TLP Tendon Modeling

TLPs are floating offshore structures anchored to the seabed by vertical tendons maintained under high pre-tension. Their dynamic behavior results from the coupled interaction between hydrodynamic loads, tendon elasticity, and platform motion. Understanding these vibration characteristics is essential for ensuring structural integrity. Over the past decades, several studies have employed beam-based idealizations to represent the structural behavior of TLP components, aiming to simplify the analysis while retaining the essential dynamic characteristics of the system. Among the pioneering works, Adrezin and Benaroya (1999) developed an analytical model describing the stochastic response of a TLP subjected to irregular sea states.

In their formulation, the platform was treated as a rigid cylinder connected to a single elastic tendon represented by an Euler-Bernoulli beam. The study demonstrated that neglecting the tendon's flexural stiffness can significantly underestimate the overall dynamic response of the system, particularly in the surge motion. Their results highlighted the importance of including tendon dynamics in coupled platform analyses to obtain realistic estimates of vibration amplitudes and natural frequencies.

A broader theoretical overview is presented by Chakrabarti (2005), where the author systematically discussed the assumptions and applicability of beam theory for representing TLP components. The author emphasized that while Euler-Bernoulli beam models provide valuable insight for linear vibration regimes, more complex behaviors, such as coupling between pitch and surge or hydrodynamic nonlinearity, require hybrid formulations combining beam and rigid-body dynamics. This distinction is critical for accurately predicting the global response of the platform when subjected to extreme environmental loading conditions.

Building on that premise, Oyejobi *et al.* (2018) investigated the dynamic coupling between the platform and its tendons through numerical simulations based on FEM. The tendons were modeled as tensioned beam elements, capturing geometric stiffness and hydrodynamic damping effects. The results confirmed that tendon flexibility and tension variations play a critical role in the system's dynamic amplification and in the distribution of stresses along the anchoring members. This finding reinforced the relevance of beam-based models as reliable tools for preliminary design and vibration prediction in TLPs.

The study by Jahangiri *et al.* (2016), proposed a diagnostic approach grounded in the vibration analysis of a piezoelectric energy harvesting system to identify structural damages,

specifically reductions in stiffness and Young's modulus. Their results showed that even small deviations in modal properties could be effectively detected using fuzzy classification, reinforcing the practical importance of accurate dynamic modeling using beam analogies for TLP monitoring.

Miranda Jr. and Santos (2017) investigated the forced response and band structure of a 1D phononic crystal beam composed of steel and polyethylene, modeled using the EBT. The study employed a comprehensive comparative analysis of FEM, Spectral Element, and Wave-based formulations, alongside Plane Wave Expansion methods. The results demonstrated that the Improved Plane Wave Expansion method yields superior convergence compared to conventional approaches and experimentally validated the existence of low-frequency Bragg-type band gaps, indicating the potential of these periodic structures for effective vibration management.

More recent research has explored the applicability of EBT beams to the modeling of TLPs. Ramos Junior, Abreu, and Hoefel (2022) performed a comparative study on the free vibration of TLP tendons using Euler-Bernoulli, Rayleigh, and Timoshenko beam theories implemented through the Finite Element Method. Their analysis revealed that for slender tendons under high tension, the Euler-Bernoulli model provides sufficiently accurate results, while for shorter or stiffer tendons, rotary inertia and shear deformation effects become relevant, and the Rayleigh and Timoshenko formulations yield improved accuracy. This comparison is particularly relevant to the present research, as it validates the adoption of the EBT model as an efficient and physically consistent representation for tendon-like elements in TLP systems.

Together, these studies provide a consistent foundation for the modeling TLP dynamics, using the EBT formulation. They demonstrate that such models capture the essential features of tendon vibration and coupling effects with satisfactory accuracy, serving as a robust framework for both analytical and numerical investigations. This body of research underpins the present study's choice to employ periodic Euler-Bernoulli beams as analog models to explore vibration attenuation mechanisms and band-gap formation in offshore structures.

2.2 Application Of Metamaterials For Vibration Mitigation In TLPs

The study of wave propagation in slender and tensioned structural members provides an intermediate theoretical foundation linking global platform dynamics to localized vibration phenomena. In tendon-like components, longitudinal and flexural waves can propagate along the structure, and their reflection, transmission, or attenuation depend strongly on geometric discontinuities and boundary conditions (Rao, 2007; Meirovitch, 2001).

In the context of offshore systems, the dynamic behavior of cables, risers, and tendons

is frequently analyzed using tensioned beam models, which emphasize the critical influence of geometric stiffness on the structural response (Adrezin; Benaroya, 1999). Expanding on the mechanics of wave propagation in such beam-like structures, Maurin and Spadoni (2016) demonstrated how periodicity and deformation levels fundamentally alter wave dispersion and propagation speeds. These principles are particularly relevant to complex marine structures, where local variations in material or cross-sectional properties can act as filtering interfaces for vibration energy, a concept reinforced by the extensive work of Senjanović (2020) on ship and offshore structural dynamics.

These findings have motivated the development of passive vibration control strategies based on engineered periodicity. Local modifications, such as stiffness variations, mass inclusions, or geometric transitions, can significantly alter the propagation characteristics of mechanical waves, leading to attenuation bands where excessive vibration is effectively suppressed. This concept, derived from the physics of phononic and elastic metamaterials, has been successfully applied in structural engineering to mitigate unwanted vibrations through geometry-induced band gaps (Hussein; Leamy; Ruzzene, 2014; Miranda Jr.; Santos, 2017).

Consequently, the application of periodicity-based principles to tendon-like elements offers a promising path for vibration mitigation in offshore platforms. The modeling of these systems through beam theories allows the identification of frequency ranges where flexural or longitudinal waves are attenuated, establishing a direct connection between the global dynamic behavior of TLPs and the localized wave phenomena explored in periodic structures.

Given the sensitivity of these slender components to dynamic excitation, standard structural damping often proves insufficient. To address this, recent research has turned to the concept of periodic structures. A periodic structure is defined as a physical system exhibiting repeating spatial patterns, which may manifest in its constituent materials, internal geometry, or boundary conditions. Historically, structural engineering has explored this concept in systems such as aircraft fuselages, ship hulls, and heat exchanger tube banks, where periodic structure theory simplified the analysis of dynamic response and reduced computational costs. In these classical applications, it was observed that such structures behave as band-pass filters, efficiently radiating noise or vibrating at specific frequencies while attenuating others. The fundamental characteristic of these systems is their unique wave propagation behavior, which can be analyzed by examining a single repeating unit cell (Hussein; Leamy; Ruzzene, 2014; Sengupta, 1980).

These systems present the distinctive ability to inhibit wave propagation within specific frequency ranges, known as band gaps. In conventional periodic structures, this phenomenon

is primarily driven by Bragg scattering, a mechanism where the spatial periodicity of material properties or geometry induces destructive interference of waves when the wavelength is comparable to the unit cell size . Building on these principles, researchers have adapted concepts from solid-state physics to design elastic metamaterials and employ ‘‘substitute continuum’’ approaches to simplify analysis. By replacing the discrete periodic structure with a continuum beam of equivalent properties, engineers can efficiently predict global dynamic behaviors, such as vibration attenuation governed by Bragg effects, using established mechanical theories (Miranda Jr.; Santos, 2017; Hussein; Leamy; Ruzzene, 2014).

To characterize the vibration suppression capabilities of realistic structures, direct numerical approaches provide a crucial alternative to classical infinite-medium theories. As employed by Domagalski (2021) in the study of periodic beams, the numerical solution of the generalized eigenvalue problem via FEM constitutes a practical framework for identifying attenuation zones. This approach allows for the detection of band gaps by analyzing the distribution and clustering of natural frequencies, where stopbands manifest as spectral ranges devoid of resonance modes. Distinguishing itself from infinite-medium assumptions, this direct analysis enables the characterization of finite systems, facilitating a straightforward comparison with global structural requirements.

Hybrid approaches are required to investigate the compelx dynamic behavior of these systems. As detailed by Reis, Souza, and Hoefel (2024) in the analysis of periodic structures, the Transfer Matrix Method combined with Bloch-Floquet theory constitutes a robust analytical framework for calculating the complex band structure of infinite periodic beams. This approach allows for the precise identification of attenuation zones (band gaps) based on the unit cell properties. Complementing this analytical insight, FEM is employed to simulate finite periodic beams, enabling the verification of mode shapes and the direct comparison of transmission spectra with practical structures. Together, these approaches form the theoretical backbone for studying periodic beams as analogs of vibration control systems.

The transition from theoretical periodic beams to practical offshore applications is well-supported in the literature. Asiri and Al-Zahrani (2014), for instance, demonstrated that introducing periodicity through passive inserts in offshore pipes effectively attenuates wave propagation, significantly reducing vibration transmission. The efficiency of these attenuation bands depends strongly on the geometric parameters of the unit cell. As shown by Iqbal and Kumar (2024) in parametric studies of periodic tracks, even subtle modifications in mass and stiffness distribution can lead to substantial changes in the attenuation characteristics. Within this context, the present

research introduces a geometric saturation parameter, α, to systematically investigate these sensitivities, linking the unit cell geometry directly to the band gap formation mechanisms essential for passive control in offshore structures.

Within this context, the present research evaluates the formation and location of band gaps via a saturation parameter α. This parameter changes unit cell configuration to allow for modified structural periodicity, and subsequent tuning of band gap magnitude and distribution along the frequency spectrum.

3 METHODOLOGY

The stepped Euler-Bernoulli beam formulation is developed using both analytical and FEM, highlighting its use for the analysis of periodic structures. MATLAB implementations of both methods are presented on appendices A and B.

3.1 Analytical Stepped Beam Formulation

The EBT is used to develop the formulation for a stepped beam consisting of n segments and total length L. Linear springs with stiffness coefficients k_{T1} and k_{Tn}, along with torsional springs with coefficients k_{R1} and k_{Rn}, are positioned at the ends of the element (as shown in Figure 2). Assuming one-dimensional and small magnitude deformation, the differential equation describing the motion of each segment in free vibration is given by:

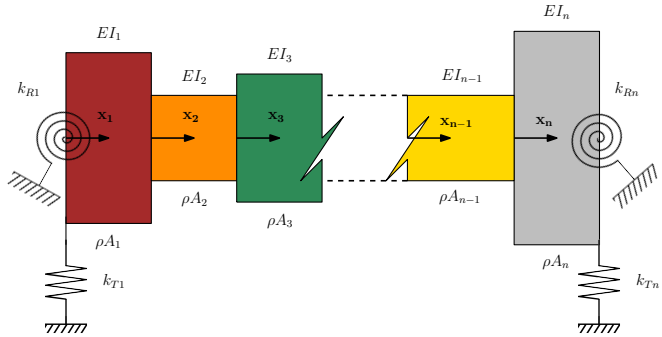
$$EI(x_i)\frac{\partial^4 w_i(x_i,t)}{\partial x_i^4} + \rho A(x_i)\frac{\partial^2 w_i(x_i,t)}{\partial t^2} = 0; \tag{1}$$

where, $EI(x_i)$ represents the transverse stiffness parameter, $\rho A(x_i)$ denotes the mass distribution, w_i is the segment-wise transverse displacement, t is the time, and i denotes an arbytrary segment number. The analytical solution can be obtained by employing the method of separation of variables on Equation 1.

The natural frequencies and mode shapes are determined using the Regula Falsi Method, as described by Koplow, Bhattacharyya, and Mann (2006). This method allows for accurate calculation of the vibration frequencies for each segment of the stepped beam. The chosen approach involves analyzing the mode shapes of each segment individually, considering their specific properties. Thus, each mode shape can be expressed as follows:

$$w(x) = A_i \sin(\beta_i x) + B_i \cos(\beta_i x) + C_i \sinh(\beta_i x) + D_i \cosh(\beta_i x), \tag{2}$$

Figure 2 – Beam with n step changes in cross sectional area.



where $\beta_i = \dfrac{\rho A_i \omega^2}{EI_i}$ represents the frequency parameter, ω represents the beam's natural frequency, and the subscript i corresponds to the segment number of a reference segment. The amplitude constants A_i, B_i, C_i, and D_i are obtained by applying the appropriate boundary conditions.

As stated by Costa Azevêdo, Vasconcelos, and Hoefel (2017), various boundary conditions, such as fixed, hinged, or movable supports at the beam ends, can be simulated by manipulating the stiffness coefficient of a spring. For this configuration, the applicable boundary conditions for n segments are:

$$\left.\frac{\partial^2 w_1}{\partial x_1^2}\right|_0 = k_{R1}\left.\frac{\partial w_1}{\partial x_1}\right|_0, \quad \left.\frac{\partial^3 w_1}{\partial x_1^3}\right|_0 = -k_{T1} w_1(0); \tag{3}$$

$$\left.\frac{\partial^2 w_n}{\partial x_n^2}\right|^{l_n} = k_{Rn}\left.\frac{\partial w_n}{\partial x_n}\right|^{l_n}, \quad \left.\frac{\partial^3 w_n}{\partial x_n^3}\right|^{l_n} = -k_{Tn} w_n(l_n). \tag{4}$$

The variations in cross-sectional area within each segment cause changes in the internal forces near the stepps (as can be seen in Figure 3), making it necessary to apply compatibility conditions about these positions. For each junction between segments i and j, the following compatibility conditions are established (Vaz, 2008):

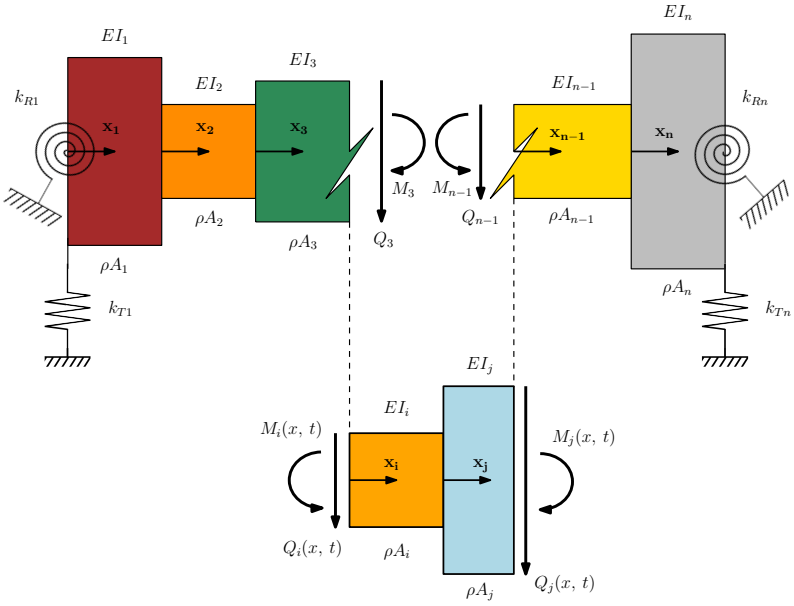
a) The vertical deformation at the end of segment i is equal to the vertical deformation at the beginning of segment j: $w_i(l_i) = w_j(0)$;

b) The slope at the end of segment i is equal to the slope at the beginning of segment j: $\left.\dfrac{\partial w_i}{\partial x_i}\right|^{l_i} = \left.\dfrac{\partial w_j}{\partial x_j}\right|_0$;

c) The resultant bending moment (M) at the end of segment i is equal to the resultant bending moment at the beginning of segment j: $E_i I_i \left.\dfrac{\partial^2 w_i}{\partial x_i^2}\right|^{l_i} = E_j I_j \left.\dfrac{\partial^2 w_j}{\partial x_j^2}\right|_0$;

d) The resultant shear (Q) at the end of segment i is equal to the resultant shear at the beginning of segment j: $E_i I_i \left.\dfrac{\partial^3 w_i}{\partial x_i^3}\right|^{l_i} = E_j I_j \left.\dfrac{\partial^3 w_j}{\partial x_j^3}\right|_0$.

Figure 3 – Compatibility of internal forces around steps.

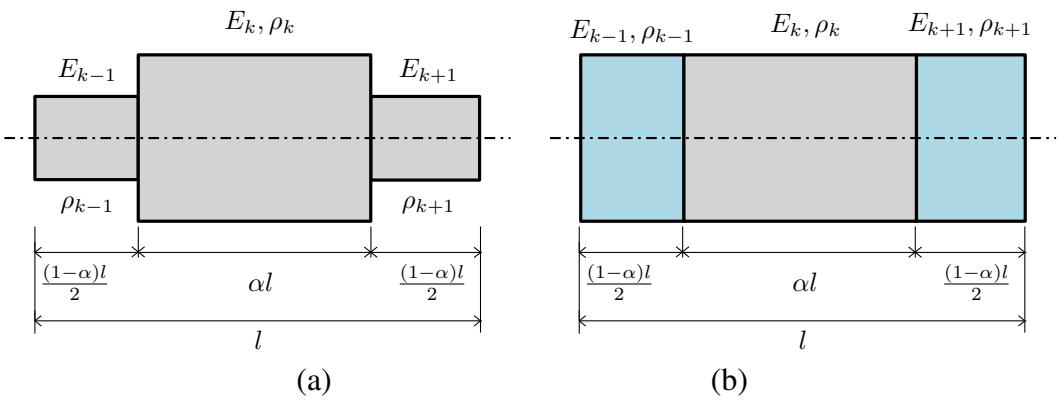


These compatibility conditions ensure the continuity of deformations, slopes, sheer forces, and bending moments along the beam, allowing for a coherent and accurate analysis of the structural behavior. The combination of boundary conditions and compatibility conditions forms a system of $4n \times 4n$ equations, whose nontrivial solution can be obtained using the Regula Falsi Method and corresponds to the natural frequencies of the system.

3.2 Modeling Periodic Structures via Stepped Beam Theory

To capture the spatial configuration of a periodic beam, a repeating unit cell of total length l is used. As shown in Figure 4, the unit cell is symmetric and composed of three segments, denoted as: left segment $(k-1)$, central segment (k), right segment $(k+1)$; where k ranges from 2 to $n-1$. Repetition of this triad allows for assembling of the final periodic structure as a stepped beam of $n = 3n_c$ segments, where n_c is the number of unit cells.

Figure 4 – Three-segment unit cell for (a) stepped and (b) bi-material beam configurations, with total length l.



The geometry of the unit cell is controlled by the saturation parameter α (Domagalski, 2021), which defines the segment length relative to the total unit cell length. Segment lengths are therefore defined, for each component of the triplet, as depicted on Figure 4.

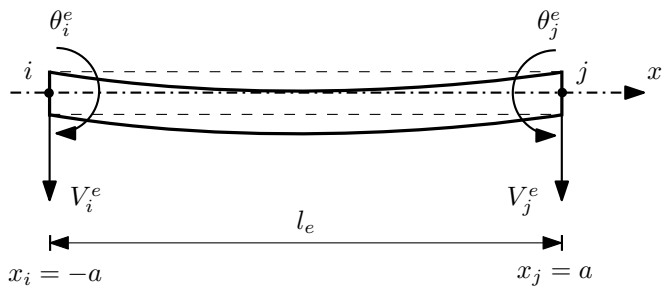
Different periodic configurations may be analysed using this approach. Geometric periodicity will be introduced by using equal areas and moments of inertia for the left and right segments, but different for the middle ($A_{k-1} = A_{k+1} \neq A_k$; $I_{k-1} = I_{k+1} \neq I_k$). The unit cell mass density and elastic modulus are kept constant ($\rho_i = \rho$; $E_i = E$).

Periodicity of material properties will be obtained by employing left and right segments with equal elastic modulus and mass density, but different to the middle one ($E_{k-1} = E_{k+1} \neq E_k$; $\rho_{k-1} = \rho_{k+1} \neq \rho_k$). While the area and moment of inertia of the unit cell are kept constant ($A_i = A$; $I_i = I$).

3.3 Modeling Periodic Structures Via FEM

An EBT beam element consists of a one-dimentional finite element, with total length l_e and length parameter a, with two nodes, and two degrees of freedom per node: the transverse deflection V_i^e and the slope θ_i^e, as shown in Figure 5. The nodal displacement vector for the element is $\{\mathbf{v}\}_e^T = [V_1^e \quad \theta_1^e \quad V_2^e \quad \theta_2^e]$.

Figure 5 – Euler-Bernoulli beam element with nodal degrees of freedom: deflection (V^e) and slope (θ^e).



The general solution of equation 1 is obtained numerically by employing a approximation function $\tilde{V}^e(x)$. The following cubic polynomial solution will be assumed:

$$\tilde{V}^e(x) = c_1 + c_2 x + c_3 x^2 + c_4 x^3 \approx V^e(x). \tag{5}$$

Determination of the four unknown constants ($c_1,\ c_2,\ c_3,\ c_4$) is achieved by enforcing nodal displacements at both ends, *i.e.*, at $x = -a$, and $x = a$:

$$V^e(-a) = V_i^e,\ \left.\frac{dV^e}{dx}\right|_{-a} = \theta_i^e; \tag{6}$$

$$V^e(a) = V_j^e,\ \left.\frac{dV^e}{dx}\right|^{a} = \theta_j^e. \tag{7}$$

This process yields the cubic Hermite shape functions, which allow the displacement field to be expressed in terms of the nodal degrees of freedom:

$$V(\xi) = [N_1(\xi) \quad N_2(\xi) \quad N_3(\xi) \quad N_4(\xi)]\{\mathbf{v}\}_e = [\mathbf{N}(\xi)]\{\mathbf{v}\}_e; \tag{8}$$

where $\xi = \dfrac{x}{l_e}$ is the non-dimensional length, $\{\mathbf{v}\}_e$ is the nodal displacement vector, and the shape functions $[\mathbf{N}(\xi)]$ are:

$$N_1(\xi) = 1 - 3\xi^2 + 2\xi^3, \tag{9}$$

$$N_2(\xi) = l_e(\xi - 2\xi^2 + \xi^3), \tag{10}$$

$$N_3(\xi) = 3\xi^2 - 2\xi^3, \tag{11}$$

$$N_4(\xi) = l_e(-\xi^2 + \xi^3). \tag{12}$$

Substitution of the displacement function 8 into the expressions of kinetic energy and strain energy, yields the elemental mass matrix $[\mathbf{m}]_e$, and elemental stiffness matrix $[\mathbf{k}]_e$:

$$T_e = \frac{1}{2}\{\dot{\mathbf{v}}\}_e^T \left(\int_{-l_e/2}^{l_e/2} \rho A [\mathbf{N}]^T [\mathbf{N}] dx \right) \{\dot{\mathbf{v}}\}_e = \frac{1}{2}\{\dot{\mathbf{v}}\}_e^T [\mathbf{m}]_e \{\dot{\mathbf{v}}\}_e, \tag{13}$$

$$U_e = \frac{1}{2}\{\mathbf{v}\}_e^T \left(\int_{l_e/2}^{l_e/2} EI_z [\mathbf{B}]^T [\mathbf{B}] dx \right) \{\mathbf{v}\}_e = \frac{1}{2}\{\mathbf{v}\}_e^T [\mathbf{k}]_e \{\mathbf{v}\}_e. \tag{14}$$

Where $[\mathbf{B}] = \dfrac{d^2}{dx^2}[\mathbf{N}]$. For elements with constant material properties (E, ρ) and uniform cross-section $(A,\ I_z)$, these integrals can be solved analytically. However, for more complex elements where these properties vary along the length, or to allow for changes of shape functions, numerical integration can be employed.

The Gauss-Legendre Quadrature is a powerful numerical integration technique. It approximates the definite integral of a function as a weighted sum of its values at specific points, known as Gauss points. For an integral over the one-dimensional domain ξ, ranging from -1 to 1, the integral of a function $f(\xi)$ is evaluated by the approximation:

$$\int_{-1}^{1} f(\xi) d\xi \approx \sum_{i=1}^{p} W_i f(\xi_i); \tag{15}$$

where p is the number of Gauss points, ξ_i are the coordinates of the Gauss points, and W_i are the corresponding weights. A change of variables $(x = l_e\xi/2)$ is used to map the element's physical domain $[-l_e/2, +l_e/2]$ to the natural coordinate domain $[-1, 1]$.

Applying this to the matrix definitions, the stiffness and mass matrices are computated numerically as:

$$[\mathbf{k}]_e = \int_{-l_e/2}^{l_e/2} EI_z [\mathbf{B}(x)]^T [\mathbf{B}(x)] dx = \frac{l_e}{2} \sum_{i=1}^{p} W_i (EI_z)_i [\mathbf{B}(\xi_i)]^T [\mathbf{B}(\xi_i)], \tag{16}$$

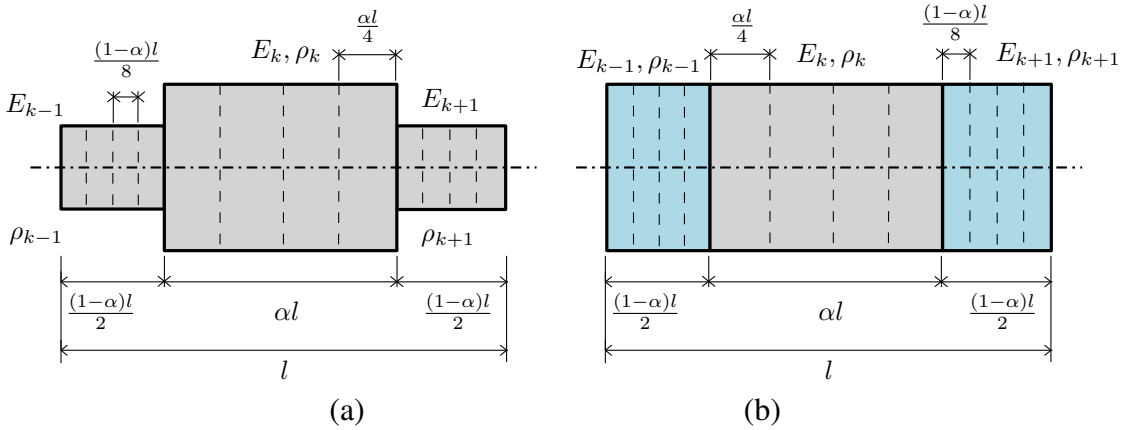
$$[\mathbf{m}]_e = \int_{l_e/2}^{l_e/2} \rho A [\mathbf{N}(x)]^T [\mathbf{N}(x)] dx = \frac{l_e}{2} \sum_{i=1}^{p} W_i (\rho A)_i [\mathbf{N}(\xi_i)]^T [\mathbf{N}(\xi_i)]; \tag{17}$$

for every element e of the discretized beam domain. The sum of which gives the local mass ($[\mathbf{M}]$) and stiffness ($[\mathbf{K}]$) matrices. Substitution of these matrices in the generalized equation of motion gives the following eigenvalue problem. By enforcing boundary conditions at both ends of the beam, the eigenvalue problem is solved for the degrees of freedom $\{\mathbf{v}\}$ in free vibration:

$$(-\omega^2 [\mathbf{M}] + [\mathbf{K}])\{\mathbf{v}\} = \{0\}. \tag{18}$$

For the analytical case, periodicity was introduced on the beam by repetition of a segment triad (Figure 4). Using FEM, a unit cell of length l is separated into three subdomains. Each subdomain is split into a number of finite elements (as can be seen in Figure 6). The outer subdomains share the same properties, while the middle subdomain presents a different configuration. Subdomain lengths, as well as element length, are defined based on the saturation parameter α. A similar approach was developed by Domagalski (2021). This collection of elements is repeated $N = 10$ times in order to create an assembly of unit cells, amounting to $n = 100$ elements, that represents a periodic beam of unitary length. Geometric and material periodicity can be introduced by properly setting element properties for each subdomain of the unit cell, as depicted in Figure 6, and in agreement with the analytical configuration (Figure 4).

Figure 6 – Unit cell meshing for (a) stepped and (b) bi-material configuration.



The local mass and stiffness matrices are then built by executing the direct sum of the elemental matrices 16 and 17 for each element relative to the generelized displacement vector $\{\mathbf{v}\}$. After which the resulting eivenvalue problem in Equation 18 is solved for the translation and rotation degrees of freedom $\{\mathbf{v}\}$. Natural frequencies ω of the periodic beam structure are obtained from this procedure.

The ‘‘direct approach’’ of frequency clustering is employed, as detailed by Domagalski (2021). By solving the eigenvalue problem for a finite assembly of N unit cells, the resulting natural frequencies are observed to group into discrete clusters corresponding to the theoretical passbands. Comparison of the clustered natural frequencies to those of a prismatic reference model allow for observation of stop bands generated by the phenomenon of Bragg scattering, *i.e.*, frequency ranges entirely devoid of resonance modes.

4 RESULTS

On this section, validations of the analytical computational routines are presented and discussed. Numerical results obtained using the FEM framework developed for the analysis of periodic EBT beams are then discussed. The later focuses on how geometric periodicity and segment saturation parameters influence the dynamic behavior, particularly the distribution of natural frequencies and the formation of band gaps.

4.1 Validations of Analytical Routines

In order to validate the computational routines developed, a numerical example is presented, proposed by Vaz (2008). It involves a Free-Free beam ($k_{T1} = k_{T2} = k_{R1} = k_{R2} = 0$), as illustrated in Figure 7. The following characteristics are considered: circular cross-sectional area (detailed below), total length $L = 321.22$ mm, mass density $\rho = 2710.3$ kg/m^3, and elastic modulus $E = 68.73$ GPa.

Three beam configurations are analyzed, each with varying cross-sectional area, where the sum of all segment lengths equals one unit:

- Beam 1: $L_1 = L_2 = L/2$ and diameters $d_1 = 14.996$ mm and $d_2 = 20.005$ mm;
- Beam 2: $L_1 = 2L/3$, $L_2 = L/3$, and diameters $d_1 = 14.967$ mm and $d_2 = 20.007$ mm;
- Beam 3: $L_1 = L_2 = L_3 = L/3$ and diameters $d_1 = 9.941$ mm, $d_2 = 14.976$ mm, and $d_3 = 20.008$ mm.

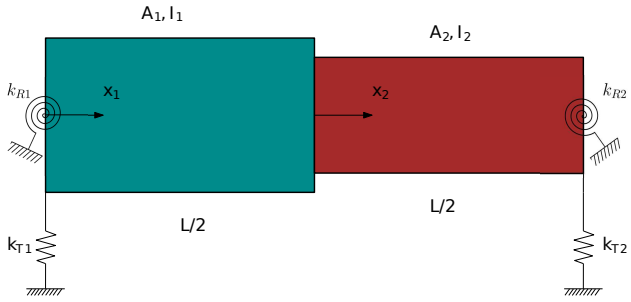
The values obtained in Table 1 show good agreement between the obtained results and the previously recorded experimental and numerical results by Vaz (2008).

Table 2 shows data relating the influence of the diameter ratio (d_2/d_1) and the natural frequencies for the one step beam on Figure 7. This investigation was previously conducted by Jang and Bert (1989b), who explored four different boundary conditions: Hinged-Hinged

Table 1 – Fundamental frequencies for three types of segmented beams.

Beam type	Vaz			Author	
	Experimental (Hz)	Numeric (Hz)	Error (%)	Numeric (Hz)	Error (%)
1	709.4	646.4	8.88	752.2	6.04
2	643.7	578.6	10.11	671.7	4.35
3	587.1	518.0	11.77	691.0	17.7

Figure 7 – Beam with one step change in cross sectional area.



($k_{T1} = k_{T2} \approx 0$, $k_{R1} = k_{R2} \rightarrow \infty$), Clamped-Clamped ($k_{T1} = k_{T2} = k_{R1} = k_{R2} \rightarrow \infty$), and Clamped-Free ($k_{T1} = k_{R1} \rightarrow \infty$, $k_{T2} = k_{R2} \approx 0$). Notably, a diameter ratio of 1 corresponds to the case of a prismatic beam. Upon analyzing the results, we find agreement between the authors' findings and the existing literature, validating the reliability of the developed routine.

Table 2 – Diameter ratios relation to natural frequencies for various boundary conditions.

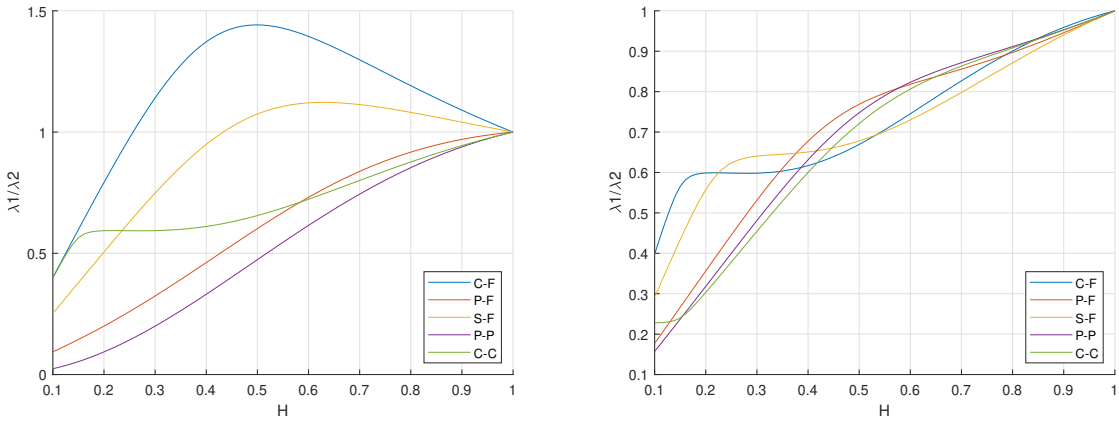
d_2/d_1	S-S	Jang	H-H	Jang	C-C	Jang	C-F	Jang
0.1	2.2435	2.2435	0.2376	0.2376	8.9123	8.9213	1.4056	1.4056
0.2	4.4875	4.4875	0.9218	0.9218	13.2701	13.2701	2.7851	2.7851
0.3	6.5555	6.5555	1.9715	1.9715	13.2811	13.2811	4.0110	4.0110
0.4	8.0567	8.0567	3.2680	3.2680	13.6651	13.6651	4.8249	4.8249
0.5	8.7794	8.7794	4.6769	4.6769	14.6697	14.6697	5.0700	5.0700
0.6	8.9907	8.9907	6.0706	6.0706	16.1756	16.1456	4.9033	4.9033
0.7	9.0572	9.0572	7.3431	7.3431	17.9092	17.9092	4.5635	4.5635
0.8	9.1843	9.1843	8.4202	8.4202	19.6133	19.6133	4.1886	4.1886
0.9	9.4500	9.4500	9.2635	9.2635	21.1232	21.1232	3.8338	3.8338
1.0	9.8696	9.8696	9.8696	9.8696	22.3733	22.3733	3.5160	3.5160

* S-S: Sliding-Sliding, H-H: Hinged-Hinged, C-C: Clamped-Clamped, C-F: Clamped-Free.

When steps are introduced to a originally prismatic beam ($d_2/d_1 = 1$), the frequency behaviour is similar to that of a continuous prismatic beam. As the step ratio approachs 0, a gradual change in dynamic behaviour is observed, building up to the elements near the step behaving independently. The stiffer section of the structure transitions from being an integral part of the structure, to acting as if a supportive element. This phenomenon is referred to as the ''local effect,'' and the corresponding mode is known as the ''local mode'' or ''component

mode'', as explained in Subramanian and Balasubramanian (1987).

Figure 8 – Natural modes number (a) one and (b) two: Frequency ratios of a stepped circular rod for various boundary conditions.



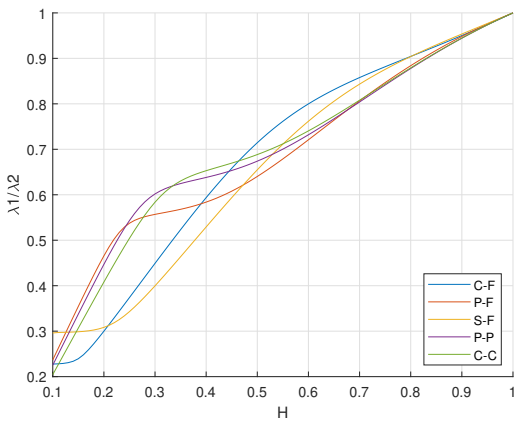
(a) (b)

To understand the effect of steps on vibration frequencies, an example presented by Subramanian and Balasubramanian (1987) was analyzed. The same beam (Figure 7) with a step in the middle was examined under various boundary conditions. Half of the beam had its dimension reduced or increased by a factor H (where $H = 4/d$). Results are presented in Table 2.

The study reveals that reducing the diameter in a specific section of the beam leads to a decrease in natural frequencies. Although the presented results focus on these three beam configurations, the developed methodology can be applied to any type of cross-sectional change, considering different elastic end support conditions.

Increases in the number of segments resulted in the highest percentage error among the three configurations. This observation aligns with previous studies conducted by Jang and Bert (1989a), Jang and Bert (1989b), and Lu *et al.* (2009), who reported similar findings.

Figure 9 – Third mode: Frequency ratios of a stepped circular rod for various boundary conditions.



Figures 8a, 8b, and 9 show the effect of changes in beam dimension on natural frequencies. The ratio λ_1/λ_2 represents the ratio between the natural frequency of the prismatic beam (λ_2) and the stepped beam (λ_1), and H represents the change in diameter of the second segment. Different boundary conditions are considered, and a comparison between the first, second, and third modes of vibration is presented, respectively.

The ratio λ_1/λ_2 can also be interpreted as the relative dynamic rigidity of the structure. Values below 1 (with $\lambda_1 < \lambda_2$), indicate a decrease in dynamic rigidity. Whereas values above 1 (with $\lambda_1 > \lambda_2$), entail an increase in beam dynamic rigidity. It is noticeable for the first vibration mode, that reduction in thickness (thinning) can, for some boundary conditions and step configurations, increase dynamic rigidity of beam structures. This effect is in agreement with Subramanian and Balasubramanian (1987), which investigated beam frequency tuning via introduction of steps in the beam's geometric configuration.

4.2 Passive Mitigation Analysis

Numerical simulations were performed for both stepped and bi-material beam configurations via FEM, considering hinged-hinged boundary conditions ($k_{T1} = k_{T2} \approx 0$, $k_{R1} = k_{R2} \to \infty$). Table 3 shows the geometric and material properties adopted for the TLP simulations (Domagalski, 2021). For simplification, both configurations consist of square sectioned segments. The stepped beam has constant height and varying widths, while material properties are kept homogeneous. The bi-material beam configuration has equal segment dimensions, but varying material properties.

Table 3 – Stepped and bimaterial configurations for Tension Leg Platform.

Bi-material Beam				
Segment	Height (h)	Width (b)	Young Modulus (E)	Density (ρ)
Middle Segment (k)	0.01 m	0.0250 m	205 GPa	7850 kg/m^3
Outter Segments ($k \pm 1$)	0.01 m	0.0250 m	69 GPa	2700 kg/m^3
Stepped Beam				
Segment	Height (h)	Width (b)	Young Modulus (E)	Density (ρ)
Middle Segment (k)	0.01 m	0.0375 m	205 GPa	7850 kg/m^3
Outter Segments ($k \pm 1$)	0.01 m	0.0250 m	205 GPa	7850 kg/m^3

The first 10 natural frequencies are calculated for a fixed saturation parameter $\alpha = 1/2$, which defines the relative length of the central segment within each unit cell. A comparison between analytical and FEM results is presented in Table 4. The two models show good agreement

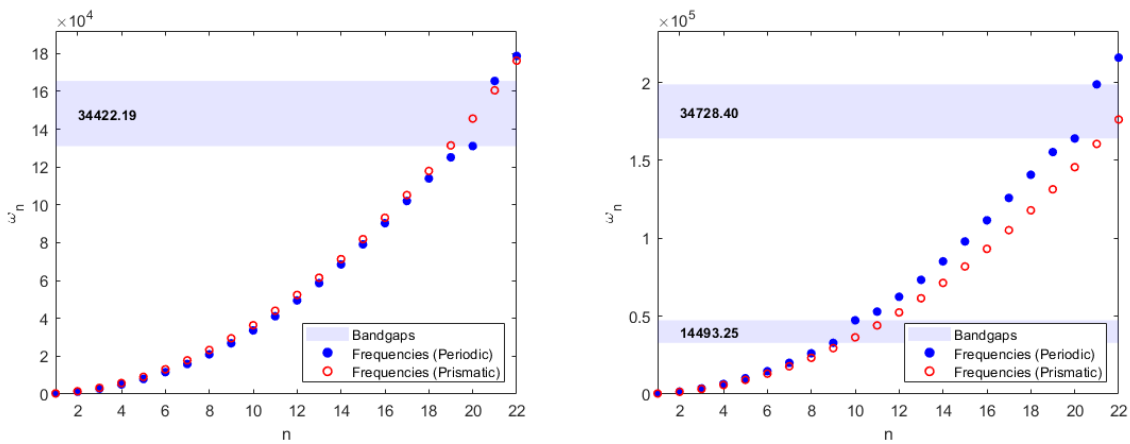
and maximum percentil error of 0.0006029% for the bi-material configuration, and 0.000688% for the stepped configuration. There is a notable increase in percentil error with increased mode numbers. A similar analysis by Domagalski (2021) yielded similar results.

Table 4 – First 10 natural frequencies (in rad/s) for stepped and bi-material configurations, with $\alpha = 1/2$.

Mode	Bi-material Beam			Stepped Beam		
	Analytic	FEM	Error (%)	Analytic	FEM	Erro (%)
1	315.2802	315.2802	0.0000026	404.4794	404.4794	0.0000014
2	1263.0599	1263.0600	0.0000028	1619.0215	1619.0216	0.0000036
3	2849.4788	2849.4789	0.0000031	3647.0420	3647.0422	0.0000048
4	5085.7799	5085.7807	0.0000149	6494.4482	6494.4494	0.0000186
5	7989.5603	7989.5633	0.0000375	10169.3888	10169.3935	0.0000465
6	11585.8390	11585.8480	0.0000779	14680.1879	14680.2025	0.0000994
7	15906.9949	15907.0185	0.0001480	20027.2895	20027.3276	0.0001903
8	20990.2931	20990.3477	0.0002602	26171.8004	26171.8893	0.0003397
9	26871.9545	26872.0700	0.0004297	32855.4483	32855.6399	0.0005831
10	33679.3825	33679.5855	0.0006029	47348.2847	47348.6104	0.0006880

To characterize the periodic effects on the TLP structure, the first 22 natural frequencies were computed for a fixed saturation parameter $\alpha = 1/2$. For the bi-material configuration, a single Bragg scattering band gap ($\Delta\omega = 34422.19$ rad/s) was detected between modes 20 and 21 (Figure 10a). Analysis of the stepped beam configuration revealed two distinct gaps: one between modes 9 and 10 ($\Delta\omega = 14493.25$ rad/s), and a second between modes 20 and 21 ($\Delta\omega = 34728.40$ rad/s) (Figure 10b).

Figure 10 – Band gaps for (a) bimaterial and (b) stepped beam configuration, with $\alpha = 1/2$.

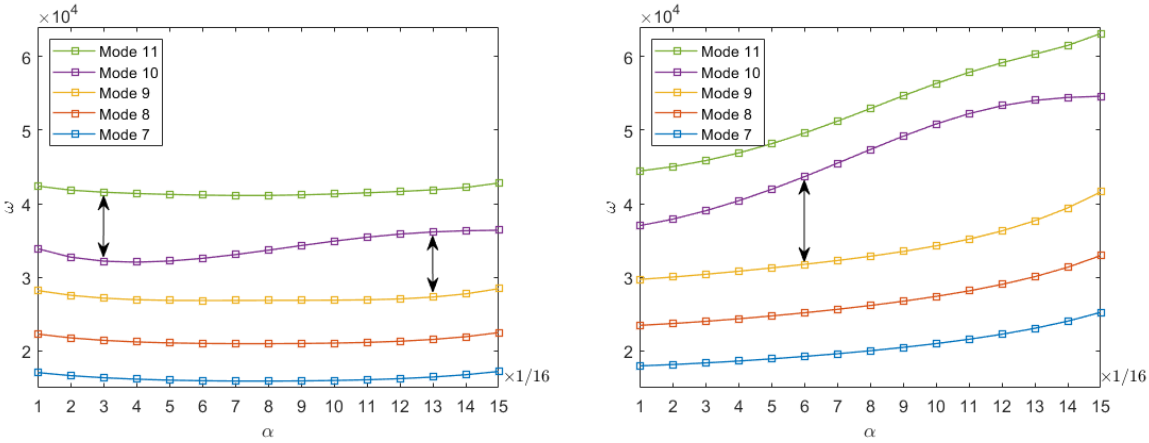


(a) (b)

Changes in the saturation parameter α lead to noticeable shifts and broadening of band-gap positions over the frequency spectrum. This result can be seen on Figures 11 and 14. The

former shows evolution of band gaps in natural frequencies between modes number 7 and 11, for $\alpha = i/16$, where $i = 1, 2, \cdots, 15$. The latter shows the same for modes 18 through 22.

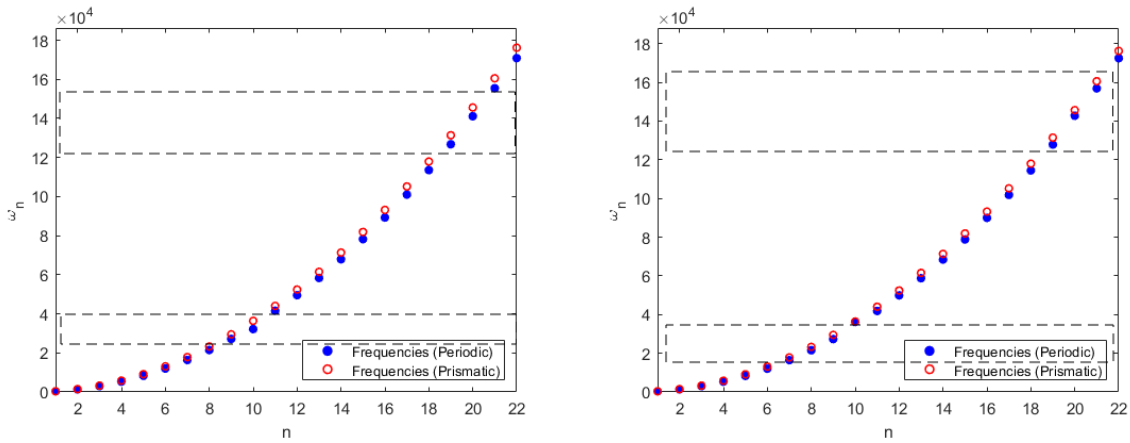
Figure 11 – Frequency gaps broadening by saturation parameter α for (a) bi-material and (b) stepped beams, for natural modes 7 through 11.



(a) (b)

Band gap formation in the bi-material configuration is analyzed in Figure 11a via the spectral spacing between modes for varying α. In the range of modes 7 to 11, the frequency gap peaks between modes 10 and 11 at $\alpha = 3/16$ ($\Delta\omega = 9363.2$ rad/s). For $\alpha > 8/16$, the primary spectral gap shifts to modes 9 and 10, peaking at $\alpha = 13/16$ ($\Delta\omega = 8846.2$ rad/s). Despite these spectral distortions, a comparison with prismatic beam frequencies (Figures 10 and 12) indicates that no effective band gaps are formed in the lower frequency spectrum.

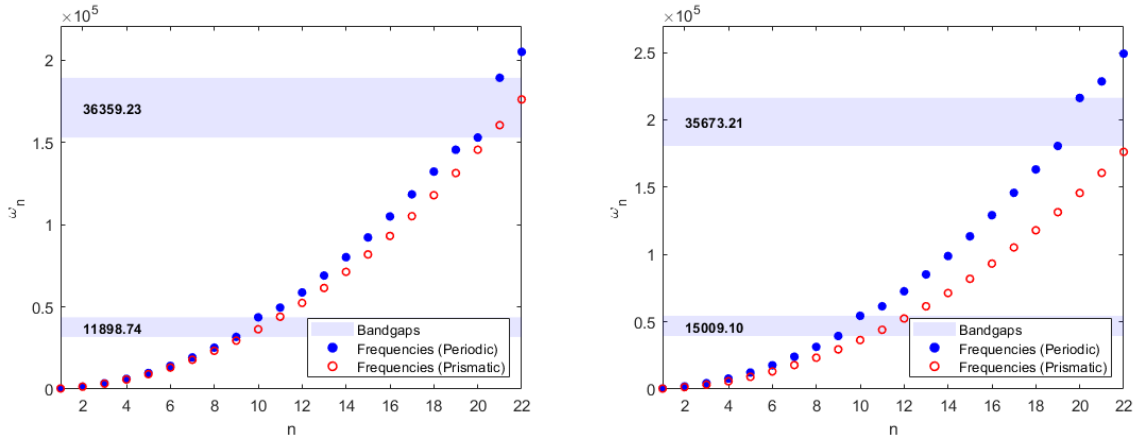
Figure 12 – Band gap absence for bi-material beam configuration on points (a) $\alpha = 3/16$ and (b) $\alpha = 13/16$.



(a) (b)

The stepped beam configuration exhibits a monotonically increasing frequency gap between modes 9 and 10 (Figure 11b), peaking at $\Delta\omega = 17044.0$ rad/s for $\alpha = 11/16$. Comparative analysis with the prismatic spectrum confirms the onset of band gaps for $\alpha \geq 6/16$ (Figure 13a).

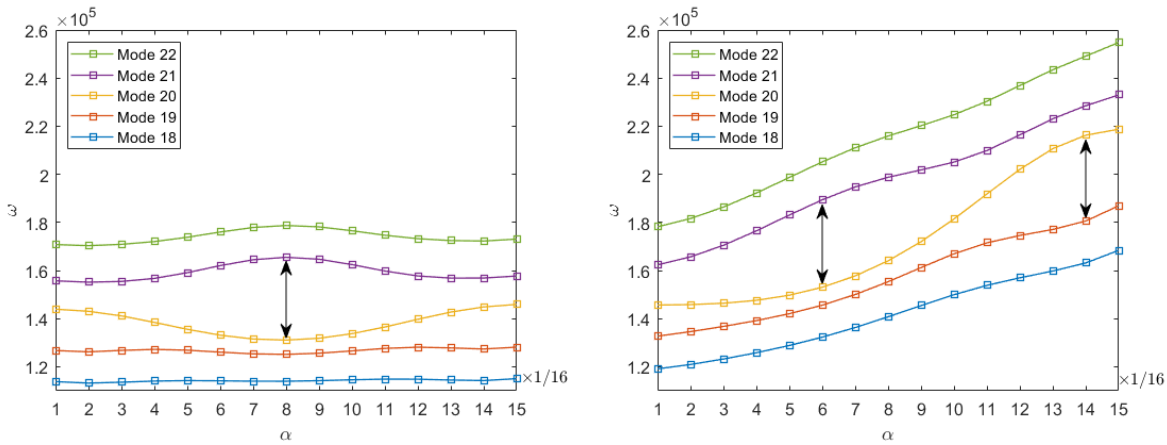
Figure 13 – Band gaps for stepped beam configuration for (a) $\alpha = 6/16$ and (b) $\alpha = 14/16$.



(a) (b)

Analysis of modes 18 through 22 in the bi-material configuration (Figure 14a) reveals minor frequency gaps between modes 19 and 20 for $\alpha \in [1/10, 3/16]$. These are subsequently shifted to modes 20 and 21. For $\alpha \in [3/16, 8/16]$, the gap broadens spectrally, reaching a maximum $\Delta\omega = 34434.56$ rad/s at $\alpha = 8/16$. The behavior is symmetric about $\alpha = 8/16$, narrowing for $\alpha \in [8/16, 15/16]$, as the mode shapes evolve and the stopband boundaries shift relative to the discrete natural frequencies. As proved from Figure 12, these spectral gaps only manifest as true band gaps for saturation parameters near $\alpha = 8/16$. Thus, material periodicity shows limited capabilities for band gap generation, resulting in restricted tunability.

Figure 14 – Band gap broadening by saturation parameter for (a) bi-material and (b) stepped beams, for natural modes 18 through 22.



(a) (b)

The stepped configuration exhibits distinct behavior for higher modes (Figure 14b). The frequency gap widens starting at $\alpha = 2/16$, reaching an absolute maximum ($\Delta\omega = 37024.0$ rad/s) at $\alpha = 7/16$ between modes 20 and 21. This gap shifts spectrally upwards for $\alpha \in [1/16, 10/16]$. However, for $\alpha > 8/16$, the attenuation zone shifts to lower mode orders, achieving a local

maximum of $\Delta\omega = 35502.0$ rad/s (at $\alpha = 14/16$) between modes 19 and 20. Analysis of Figures 11, 13, and 14 confirms that geometric periodicity induces two distinct band gaps across various saturation parameters. This establishes the stepped beam as a superior, more tunable structure compared to the bi-material configuration for vibration control in TLP.

5 CONCLUSIONS

The present study successfully analyzed the dynamic behavior of periodic structures applied to TLP tendons, demonstrating the efficacy of geometric and material periodicity as passive vibration control strategies. By modeling tendons as EBT beams, configured as both stepped and bi-material structures, a clear relationship between the unit cell geometry and the formation of frequency band gaps was established.

The implementation of FEM routines in MATLAB yielded consistent results when validated against analytical models, achieving a maximum percentile error of approximately 0.000688% for the first ten natural frequencies.

The introduction of the geometric saturation parameter α, proved to be a critical design variable for tuning attenuation zones. The parametric analysis revealed distinct behaviors for the two configurations investigated:

- Bi-material configuration: While capable of generating band gaps, such as the gap observed between modes 20 and 21, the bi-material beam showed limited tunability. The attenuation zones exhibited significant spectral migration, often traversing across mode indices as the saturation parameter varied, or required specific saturation parameters (*e.g.*, near $\alpha = 8/16$) to manifest effectively.

- Stepped configuration: Introducing geometric periodicity resulted in superior vibration mitigation capabilities. This beam type exhibited multiple, wider band gaps, notably a maximum gap observed at $\alpha = 11/16$. Analysis confirmed that geometric variations enable the creation of two distinct attenuation zones across diverse configurations, rendering the stepped beam a more tunable and efficient structure for TLP applications than its bi-material counterpart.

In summary, optimizing the saturation parameter α for a stepped beam configuration offers a promising engineering solution for passive mitigation of vibration in offshore platforms.

These findings contribute to the development of safer and more durable marine structures by effectively filtering vibrations on revelant frequency spectrum.

REFERENCES

ADREZIN, R.; BENAROYA, H. Response of a Tension Leg Platform to Stochastic Wave Forces. **Probabilistic Engineering Mechanics**, Elsevier, v. 14, p. 3–17, 1999. DOI: https://doi.org/10.1016/s0266-8920(98)00012-5.

ASIRI, Saeed A.; AL-ZAHRANI, Yousuf Z. Theoretical Analysis of Mechanical Vibration for Offshore Platform Structures. **World Journal of Mechanics**, Scientific Research, v. 4, p. 1–11, 2014. DOI: http://dx.doi.org/10.4236/wjm.2014.41001.

CHAKRABARTI, Subrata K. (ed.). **Handbook of Offshore Engineering**. 1. ed. Amsterdam: Elsevier, 2005. ISBN 978-0-08-044381-2.

COSTA AZEVÊDO, A. S. da; VASCONCELOS, A. C. A.; HOEFEL, S. dos S. Dynamic analysis of elastically supported Timoshenko beam. **Revista Interdisciplinar de Pesquisa em Engenharia**, v. 2, n. 34, p. 71–85, ago. 2017. Available from: https://periodicos.unb.br/index.php/ripe/article/view/21809.

DOMAGALSKI, Łukasz. Comparison of the Natural Vibration Frequencies of Timoshenko and Bernoulli Periodic Beams. **Materials**, MDPI, v. 14, n. 24, p. 7628, 2021. DOI: 10.3390/ma14247628.

HUSSEIN, Mahmoud I; LEAMY, Michael J; RUZZENE, Massimo. Dynamics of Phononic Materials and Structures: Historical Origins, Recent Progress, and Future Outlook. **Applied Mechanics Reviews**, v. 66, n. 4, 2014. DOI: https://doi.org/10.1115/1.4026911.

IQBAL, Mohd; KUMAR, Anil. Analysis of bending waves and parametric influence on band gaps in periodic track structure. **Materials Today: Proceedings**, Elsevier, v. 113, p. 235–239, 2024. DOI: 10.1016/j.matpr.2023.08.364.

JAHANGIRI, Vahid *et al.* TLP Structural Health Monitoring Based on Vibration Signal of Energy Harvesting System. **Latin American Journal of Solids and Structures**, Scielo, v. 13, n. 5, p. 897–915, 2016. DOI: http://dx.doi.org/10.1590/1679-78252282.

JANG, S K; BERT, C W. Free vibration of stepped beams: Exact and numerical solutions. **Journal of Sound and Vibration**, v. 130, n. 2, p. 342–346, 1989a.

JANG, S K; BERT, C W. Free vibration of stepped beams: Higher mode frequencies and effects of steps on frequency. **Journal of Sound and Vibration**, v. 132, n. 1, p. 164–168, 1989b.

KANDASAMY, Ramkumar *et al.* A review of vibration control methods for marine offshore structures. **Ocean Engineering**, Elsevier, v. 127, p. 279–297, 2016. DOI: http://dx.doi.org/10.1016/j.oceaneng.2016.10.001.

KOPLOW, M A; BHATTACHARYYA, A; MANN, B P. Closed form solutions for the dynamics response of Euler-Bernoulli beams with step changes in cross section. **Journal of Sound and Vibration**, v. 295, p. 214–225, 2006.

LU, Z R *et al.* Vibration analysis of multiple-stepped beams with the composite element model. **Journal of Sound and Vibration**, v. 322, p. 1070–1080, 2009.

MAURIN, Florian; SPADONI, Alessandro. Wave propagation in periodic buckled beams. Part I: Analytical models and numerical simulations. **Wave Motion**, Elsevier, v. 66, p. 190–209, 2016. DOI: `https://doi.org/10.1016/j.wavemoti.2016.05.008`.

MEIROVITCH, Leonard. **Fundamentals of Vibrations**. [*S. l.*]: McGraw-Hill, 2001.

MIRANDA JR., E. J. P. de; SANTOS, J. M. Campos dos. Flexural Wave Band Gaps in Phononic Crystal Euler-Bernoulli Beams Using Wave Finite Element and Plane Wave Expansion Methods. **Materials Research**, v. 20, Suppl. 2, 2017. DOI: `https://doi.org/10.1590/1980-5373-MR-2016-0877`.

OYEJOBI, D. *et al.* Investigation of Tendon Dynamics Effects on Tension Leg Platform Response in Random Seas. **Proceedings of the Institution of Mechanical Engineers Part M Journal of Engineering for the Maritime Environment**, v. 233, n. 4, 2018. DOI: `https://doi.org/10.1177/1475090218811718`.

RAMOS JUNIOR, José Pereira; ABREU, Brendon Menezes de; HOEFEL, Simone dos Santos. Free Vibration Analysis of a Tension Leg Platform Tendon. *In:* ABCM. ANAIS do XI Congresso Nacional de Engenharia Mecânica (CONEM 2022). Teresina, PI, Brazil: [*s. n.*], 2022.

RAO, Singiresu S. **Vibration of Continuous Systems**. [*S. l.*]: John Wiley & Sons, 2007.

REIS, J.; SOUZA, I.; HOEFEL, S. Flexural wave propagation in a locally resonant beam. *In:* ANAIS do XII Congresso Nacional de Engenharia Mecânica (CONEM 2024). Natal, RN, Brazil: [*s. n.*], 2024.

SENGUPTA, G. Vibration of Periodic Structures. **The Shock and Vibration Digest**, v. 12, n. 3, p. 17–31, 1980. DOI: `https://doi.org/10.1177/058310248001200303`.

SENJANOVIĆ, Ivo. **Ivo Senjanović: Scientific Contribution to the Development of Ship and Offshore Structures**. Ed. by Hadžić N., Vladimir N. and Malenica Š. Zagreb, Croatia: University of Zagreb, Faculty of Mechanical Engineering and Naval Architecture, 2020. Monograph published on the occasion of his 70th birthday. ISBN 978-953-7738-70-9.

SUBRAMANIAN, G; BALASUBRAMANIAN, T S. Beneficial effects of steps on the free vibration characteristics of beams. **Journal of Sound and Vibration**, v. 118, n. 3, p. 555–560, 1987.

VAZ, J C. **Análise do Comportamento Dinâmico de uma Viga de Euler-Bernoulli Escalonada com Apoios Elasticamente Variáveis**. 2008. MA thesis – Graduate Program in Mechanical Engineering, Federal University of Itajubá, Itajubá, Brasil.

APPENDIX A– ANALYTICAL MATLAB ROUTINES

Routine 1 – Beam Setup.

```
%{
Beam data main file

Analised Beam Structure: Euler-Bernoulli stepped beam.
Torsional springs on ends (testing) linear springs on ends. (testing)
Axial load distribution. (seg const / not working)
Dynamic analysis: changes in segment mass distribution. (prototype)

Supported Boundary Conditions: FF, CC, PP, CF, CP, CS, (validated)
                               EF : Elastic-Free (testing).

Gives beam data to Regula Falsi routine [falsaposicao.m].

Output:
Frequencies: dimentional [rad/s], non-dimentional [-],
             cyclic frequencies [Hz].

Recommendations:
- Recalculate system matrix using a dimentional approach, instead of
  non-dimentional. Then, axial load should work.
- Otherwise check non-dimentionalization of axial load for matrix issues.
- Switching to struct-based input could be useful but code performance has
  to be assessed.

By h4lfling.
%}

close all; clear;
tic;
format short;

% %{
%-------------------------- PRISMATIC BEAM DATA --------------------------%
% Number of modeshapes
modos = 8;

% Case validation switch
validacao = 0;
simcompi = 1;

% Validation 1: Back-check against paper
if validacao==1
    % for r=cat(2,0.1:0.1:1,1.5:.5:2),viga,end
    d1 = 0.05;
```

```
    %{
    r = 0.8; diameter = [d1 d1*r];
    %}
    % %{
    n_seg = 3;
    % beam.n_step = 3;
    diameter = [d1*r d1 d1*r];
    % beam.d = [d1*r d1 d1*r]; % <<<<<<<<<<<<<<<<<
    di = diameter;
    % %}
    % .0217    0.7522    2.1984

% Periodic Beam Analysis ------------------------------------------------
elseif simcompi==1
    % Param. de periodicidade
    alpha_artigo = 1-(4/8);
    LS2 = (alpha_artigo/2);
    LS1 = 1-2*LS2;

    % Param. gerais
    n_c = 10; L0 = 1; n_seg = 3*n_c;

    % Caso: Bimaterial || Bigeometria
    bimat = 0;

    if bimat==0  % Caso: Bigeometria
        % Prop. material
        rho_c = [7850 7850 7850];  % kg/m^3
        Ec = [205e9 205e9 205e9];  % Pa = N/m^2

        % Prop. geomÃ©tricas
        d1 = 0.01*ones(1,3);          % Altura [m]
        d2 = [0.0250 0.0375 0.0250];  % Base   [m]

    else % Caso: Bimaterial
        % Prop. material
        rho_c = [2700 7850 2700];  % kg/m^3
        Ec = [69e9 205e9 69e9];   % Pa = N/m^2

        % Prop. geomÃ©tricas
        d1 = 0.01*ones(1,3);    % Altura [m]
        d2 = 0.0250*ones(1,3);  % Base   [m]
    end
    Lc = [LS2, LS1, LS2] * L0/n_c; % Comp. CÃ©lula

% Other case ------------------------------------------------------------
elseif validacao==0
```

```
    n_seg = 9;
    diameter = ones(1,n_seg) * 0.05; % [m]
    % Total beam legnth
    L0 = 2; % 321.22/1000;     % [m]
    % Segment length
    L = ones(1,n_seg)*L0/n_seg;        % [m]   ( per step )
    rho = 2710.3; %7850;           % kg/m^3
    E = 6.873e+10; %2.068e+11; % ; % Pa = N/m^2

    % beam.d = ones(1,n_seg) * 0.05; % [m] % <<<<<<<<<<<<<<<<<
    % diameter = [ 0.005 0.006 0.009 ];
    % diameter = [14.996/1000 20.005/1000];
    % diameter = [14.967/1000 20.007/1000];
    % diameter = [ 9.941/1000 14.976/1000 20.008/1000 ];
    % diameter = [ diam diam/(2^(1/4)) ];
end

% Geometric and Material Properties -------------------------------------
if simcompi==1
    Area = repmat(d1.*d2, 1, n_c);    % m^2
    I = repmat(d1.*d2.^3/12, 1, n_c); % m^4
    E = repmat(Ec, 1, n_c);           % m^4
    rho = repmat(rho_c, 1, n_c);      % m^4
    EI = E .* I;                      % N.m^2
    RhoA = rho .* Area;
    L = repmat(Lc, 1, n_c);           % m
else
    I = (pi/64).*diameter.^4;         % m^4
    Area = pi .* (diameter/2).^2;     % m^2
    EI = E .* I;                      % N.m^2
    RhoA = rho .* Area;               % kg/m
end

test = 0; % test switch for BC

Li = L./L0;          % non-dimentional relative segment length (=l_i/L0)
a = zeros(1,n_seg);  % non-dimentional axial force (keep at 0)
%-------------------------------------------------------------------------%

%------------------------- BOUNDARY CONDITIONS --------------------------%
% Boundary Condition selection according to options below
%
% 1 = [FF] | 2 = [CC] | 3 = [PP] | 4 = [CF] | 5 = [CP] | 6 = [CS]
% 7 = [elastic-F]
%
BC = 3;
boundaryConditions();
```

```
%-------------------------------------------------------------------------%

%-------------------------- SINGLE FREQUENCY ----------------------------%
getFrequencies();
%-------------------------------------------------------------------------%

%------------------------- DYNAMIC MODAL ANALYSIS -------------------------%
% dynamicAnalysis();
%-------------------------------------------------------------------------%
toc;
```

Routine 2 – Boundary Conditions Setup.

```
%{
Boundary Conditions configuration file. Validated against Wu, 2013.

OBS: {R1 R2 T1 T2} are NOT spring constants. they equal EI/K, where K is
the spring constant.

By h4fling.
%}

% Supported Boundary Conditions: FF, CC, PP, CF, CP, CS, EF (testing).
% 1 = [FF] | 2 = [CC] | 3 = [PP] | 4 = [CF] | 5 = [CP] | 6 = [CS]
% 7 = [elastic-F]
% when BC=7, define spring parameter k1 beforehand

if BC==1                % Free-Free
    % n_seg momentum parameter
    R1 = 1e9*L(1);        % N.m/rad
    R2 = 1e9*L(n_seg);    % N.m/rad
    % n_seg sheer stress
    T1 = 1e9*L(1)^3;      % N/m
    T2 = 1e9*L(n_seg)^3;  % N/m

elseif BC==2            % Clamped-Clamped
    % n_seg momentum parameter
    R1 = 1e-8*L(1);       % N.m/rad
    R2 = 1e-8*L(n_seg);   % N.m/rad
    % n_seg sheer stress
    T1 = 1e-8*L(1)^3;     % N/m
    T2 = 1e-8*L(n_seg)^3; % N/m

elseif BC==3            % Pinned-Pinned
    if simcompi==0
        R1 = 1e8*L(1);        % N.m/rad
        R2 = 1e8*L(n_seg);    % N.m/rad
    elseif bimat=1
```

```
        R1 = 1e6*L(1);         % N.m/rad
        R2 = 1e6*L(n_seg);     % N.m/rad
    elseif bimat=0
        R1 = 1e4*L(1);         % N.m/rad
        R2 = 1e4*L(n_seg);     % N.m/rad
    end
    % n_seg sheer stress
    T1 = 1e-8*L(1)^3;      % N/m
    T2 = 1e-8*L(n_seg)^3; % N/m

elseif BC==4             % Clamped-Free
    R1 = 1e-8*L(1);        % N.m/rad
    R2 = 1e8*L(n_seg);     % N.m/rad
    % n_seg sheer stress
    T1 = 1e-8*L(1)^3;      % N/m
    T2 = 1e8*L(n_seg)^3;   % N/m

elseif BC==5             % Clamped-Pinned
    R1 = 1e-8*L(1);        % N.m/rad
    R2 = 1e8*L(n_seg);     % N.m/rad
    % n_seg sheer stress
    T1 = 1e-8*L(1)^3;      % N/m
    T2 = 1e-8*L(n_seg)^3; % N/m

elseif BC==6             % Campled-Sliding
    % n_seg momentum parameter
    R1 = 1e-8*L(1);         % N.m/rad
    R2 = 1e-8*L(n_seg);     % N.m/rad
    % n_seg sheer stress
    T1 = 1e-8*L(1)^3;      % N/m
    T2 = 1e8*L(n_seg)^3;   % N/m

elseif BC==7             % Elastic-Free
    R1 = k1*L(1);          % N.m/rad
    R2 = 1e9*L(n_seg);     % N.m/rad
    % n_seg sheer stress
    T1 = k1*L(n_seg)^3;    % N/m
    T2 = 1e9*L(n_seg)^3;   % N/m
end
```

Routine 3 – Analytical Beam: Frequency Display.

```
%{
Frequency Routine
- Calculates single case frequency.
by h4lfling.
%}
```

```
if validacao==1
    format compact
    freq_dim = falsaposicao(modos,L0,R1,R2,T1,T2,EI,RhoA,Li,a); % [rad/s]
    freq_adim = ( L0^4 * freq_dim.^2 * RhoA(1) / EI(1) ).^(1/4); % [-]

%     disp([sprintf('%.3f',r),'    ',sprintf('%.4f',freq_adim(1)^2)])

    %{
    disp([sprintf('%s','  d1'),...
        sprintf('%s',' |  d2')...
        sprintf('%s',' |  d3')...
        sprintf('%s',' |  freq1')]);

%         sprintf('%.3f',diameter(3)),'|',...
    %}
%     disp(freq_adim)
    disp([sprintf('%.3f',r),'|',...
        sprintf('%.3f',diameter(1)),'|',...
        sprintf('%.3f',diameter(2)),'|',...
        sprintf('%.4f',freq_adim(1)^2)]);
%     %}

elseif validacao==2
    format compact
    freq_dim = falsaposicao(modos,L0,R1,R2,T1,T2,EI,RhoA,Li,a); % [rad/s]
    freq_adim = ( L0^4 * freq_dim.^2 * RhoA(1) / EI(1) ).^(1/4); % [-]

%     disp([sprintf('%.3f',r),'    ',sprintf('%.4f',freq_adim(1)^2)])

    %{
    disp([sprintf('%s',' k1 '),'|',...
        sprintf('%s',' freq1 ')]);
    %}
    disp([sprintf('%.3f',k1),' | ',...
        sprintf('%.4f',freq_adim(1)^2)]);

elseif test==1
    format compact
    freq_dim = falsaposicao(modos,L0,R1,R2,T1,T2,EI,RhoA,Li,a); % [rad/s]
    freq_adim = ( L0^4 * freq_dim.^2 * RhoA(1) / EI(1) ).^(1/4); % [-]

%     disp([sprintf('%.3f',r),'    ',sprintf('%.4f',freq_adim(1)^2)])

    %{
    disp([sprintf('%s','  d1'),...
        sprintf('%s',' |  d2')...
        sprintf('%s',' |  d3')...
```

```
        sprintf('%s',' |  massa')...
        sprintf('%s',' |  freq1')]);
    %}
    disp([sprintf('%.3f',diameter(1)),'|',...
        sprintf('%.3f',diameter(2)),'|',...
        sprintf('%.3f',diameter(3)),'|',...
        sprintf('%.4f',massa),'    ',...
        sprintf('%.4f',freq_adim(1))]);
%     %}

else
    format short;disp("Working...");
    freq_dim = falsaposicao(modos,...
        L0,R1,R2,T1,T2,EI,RhoA,Li,a); % [rad/s]
    freq_hz = freq_dim/(2*pi);

    % Assuming EI(1) and RhoA(1) as reference segment
    B_ = sqrt((RhoA(1) * L0^4) / EI(1));
    % Calculate normalized eigenvalues (equivalent to FEM's 'b')
    b_normalized = (freq_dim * B_).^2;
    % Dispersion relation parameter
    freq_adim = (L0^4 * freq_dim.^2 * RhoA(1) / EI(1)).^(1/4);

    disp("=== Frequency Comparison ===");
    disp("Natural frequencies [rad/s]:");
    fprintf('%.4f, %.4f\n',freq_dim);
    disp("Natural frequencies [Hz]:");
    fprintf('%.4f, %.4f\n',freq_hz);
    disp("Dispersion parameter Î²:");
    fprintf('%.4f, %.4f\n',freq_adim);
end
```

Routine 4 – Regula Falsi Method.

```
%{
    Rotina da Falsa Posição.
    By slepton, h4lfling.

    Adaptado para estrutura de foguete:
        Recebe parâmetros da simulação de [viga.m]
        Passa parâmetros para rotina da matriz [func.m]

    Modificado Março/2023:
        - raiz = 'a funçao nao apresenta mais raizes' torna-se raiz = ''.
        - Valor inicial modificado de [a = 0] para [a = 0.01].
    Modificado Jan/2025:
        - Precisão modificada de 0.000001 (1e-6) para 1e-5.
        - Valor inicial modificado de [a=0.01] para [a=0.1].
```

```
  Modificado Abr/2025:
    - Valor inicial mudado para 'a=1'.
        - Passo mudado para 'passo=1'.
    Bugs identificados:
        - Possível duplicação do cálculo de raíz ao redor de x=0.
%}

function [raiz] = falsaposicao(quantmax,L0,R1,R2,T1,T2,EI,RhoA,Li,A)

clear c;

%dados iniciais
quant = 0; %contador para a quantidade de raizes
a = 1; %primeiro ponto da funçao
passo = 1; %aumentar a divisao para que pegue resultados menores
b = passo; %segundo ponto
raiz = zeros(1,quantmax);
%laço para repetir o programa ate achar todas as raizes pedidas

while quant < quantmax
    tol = 1e-4;  % tolerância de erro
    %funçoes de a e b
    fa = func(a,L0,R1,R2,T1,T2,EI,RhoA,Li,A);
    fb = func(b,L0,R1,R2,T1,T2,EI,RhoA,Li,A);
    teste = bolzano(fa,fb,1); %uso de bolzano de fa, fb e o contador de tentativas
    if teste == 0 %contagem maior que 10000
        raiz = 'a funçao nao apresenta mais raizes';
        break %programa acaba
    elseif teste == 1
        raiz(quant) = falsa(a,b,tol); % faixa de erro
        % raiz(quant) = falsa(...
    end
end

%algoritimo da falsa posiçao
    function [x] = falsa(a2,b2,erro)
        fa2 = func(a2,L0,R1,R2,T1,T2,EI,RhoA,Li,A);
        fb2 = func(b2,L0,R1,R2,T1,T2,EI,RhoA,Li,A); %calculo das funçoes

        x = (a2-(b2-a2)*fa2/(fb2-fa2));
        %laço que se repetira ate ser menor que o erro
        while abs(x - a2) >= erro
            a2 = x; %aplicando o novo a, pois b é fixo
            fa2 = func(a2,L0,R1,R2,T1,T2,EI,RhoA,Li,A);

            % Calculo da falsa posiçao
            x = (a2-(b2-a2)*fa2/(fb2-fa2));
```

```
            if fb2 == 0 %caso a raiz esteja exatamente em b
                x = b2;
            end
        end
        quant = quant + 1;
        a = b + passo; %a aumenta para o intervalo maximo sempre ser igual ao passo
        b = b + 2*passo; %aumentando b para testar novamente
    end

%funçao de bolzano
    function [resultado] = bolzano(fa, fb, contmax) %contmax é o numero de
    tentativas

        while fa*fb > 0

            a = b; %a aumenta para o intervalo maximo sempre ser igual ao passo
            b = b + passo; %aumentando b para testar novamente
            fa = func(a,L0,R1,R2,T1,T2,EI,RhoA,Li,A); %calculando a nova funcao de a
     para o teste do while
            fb = func(b,L0,R1,R2,T1,T2,EI,RhoA,Li,A); %calculando a nova funcao de b
     para o teste do while

            contmax = contmax+1; %laço de que segue ate o maximo

            if contmax == 1e20
                break
            end
        end

        if contmax == 1e20 %maximo de vezes que o computador vai tentar

            resultado = 0;
        else

            resultado = 1;
        end
        contmax = 0;
    end
end
```

Routine 5 – Characteristic Matrix Function.

```
% Matrix routine for Rocket Beam natural frequencies.
% Receives beam data from Regula Falsi routine [falsaposicao.m].
%
% Rocket Structure:
%     Stepped Euler-Bernoulli ( n steps ) [works for traditional BC].
%     torsional spring on both ends [works for traditional BC].
```

```
%     linear spring on both ends [works for traditional BC].
%     axial load distribution ( segment constant ) [not working].
%
% Output: matrix determinant, returns to [falsaposicao.m].
% By h4lfling

function [determinante] = func(omega,L0,R1,R2,T1,T2,EI,RhoA,Li,a)

% { R1 R2 T1 T2 L0 omega } are constant numbers
% { EI RhoA Li a         } are vectorial

n = length(EI); % quant. of segments

matriz = zeros(4*n,4*n);

lambda_seg = zeros(1,n);
lambX1 = lambda_seg; lambX2 = lambda_seg;

S = zeros(1,n); C = S; SH = S; CH = S;
EIR = zeros(1,n-1);
for i=1:(n-1)
    EIR(i) = EI(i+1)/EI(i);
end
for i=1:n
    lambda_seg(i) = ( RhoA(i)*L0^4*omega^2/EI(i) )^(1/4);
    lambX1(i) = ( 0.5 * ( (a(i)^2+4*lambda_seg(i)^4)^(1/2) - a(i) ) )^(1/2);
    lambX2(i) = ( 0.5 * ( (a(i)^2+4*lambda_seg(i)^4)^(1/2) + a(i) ) )^(1/2);
    S(i) = sin( lambX2(i) * Li(i) );
    C(i) = cos( lambX2(i) * Li(i) );
    SH(i) = sinh( lambX1(i) * Li(i) );
    CH(i) = cosh( lambX1(i) * Li(i) );
end

%--------------------- BOUNDARY CONDITIONS ---------------------%
flexao_0 = L0 * [   lambX2(1);
                    (lambX2(1)^2 + a(1))*R1;
                    lambX1(1);
                  - (lambX1(1)^2 - a(1))*R1    ]';

cort_0 = [ -(lambX2(1)^3 + lambX2(1)*a(1))*T1;
            1;
           -(lambX1(1)*a(1) - lambX1(1)^3)*T1;
            1                                  ]';

flexao_n = -L0*[ -R2*S(n)*( lambX2(n)^2  + a(n) ) - lambX2(n)*C(n);
                -R2*C(n)*( lambX2(n)^2  + a(n) ) + lambX2(n)*S(n);
                 R2*SH(n)*( lambX1(n)^2 - a(n) ) - lambX1(n)*CH(n);
```

```
                R2*CH(n)*( lambX1(n)^2 - a(n) ) - lambX1(n)*SH(n) ]';

cort_n = [ -T2* C(n)*( lambX2(n)^3 + a(n)*lambX2(n) ) +  S(n);
           T2* S(n)*( lambX2(n)^3 + a(n)*lambX2(n) ) +  C(n);
           T2*CH(n)*( lambX1(n)^3 - a(n)*lambX1(n) ) + SH(n);
           T2*SH(n)*( lambX1(n)^3 - a(n)*lambX1(n) ) + CH(n) ]';

matriz([1,2],1:4) = [flexao_0; cort_0];
matriz([3,4],(4*n-3):(4*n)) = [flexao_n; cort_n];
%-----------------------------------------------------------------%

%------------------- COMPATIBILITY CONDITIONS ------------------%
for i=1:(n-1)

    compt1 = [ S(i) C(i) SH(i) CH(i) 0 -1 0 -1 ];

    % poderia ser (i) de 4 a 8?
    compt2 = L0*[  lambX2(i)*C(i);
                  -lambX2(i)*S(i);
                   lambX1(i)*CH(i);
                   lambX1(i)*SH(i);
                  -lambX2(i+1);
                   0;
                  -lambX1(i+1);
                   0 ]';

    compt3 = L0^2*[ - S(i) *( lambX2(i)^2 + a(i) );
                    - C(i) *( lambX2(i)^2 + a(i) );
                      SH(i)*( lambX1(i)^2 - a(i) );
                      CH(i)*( lambX1(i)^2 - a(i) );
                      0;
                      EIR(i)*(  lambX2(i+1)^2 + a(i+1) );
                      0;
                      EIR(i)*( -lambX1(i+1)^2 + a(i+1) ) ]';

    compt4 = L0^3*[ - C(i) *( lambX2(i)^3 + a(i)*lambX2(i) );
                      S(i) *( lambX2(i)^3 + a(i)*lambX2(i) );
                      CH(i)*( lambX1(i)^3 - a(i)*lambX1(i) );
                      SH(i)*( lambX1(i)^3 - a(i)*lambX1(i) );
                      EIR(i)*( lambX2(i+1)^3 - a(i+1)*lambX2(i+1) );
                      0;
                    - EIR(i)*( lambX1(i+1)^3 + a(i+1)*lambX1(i+1) );
                      0 ]';
    matriz([1+4*i,2+4*i,3+4*i,4+4*i],(1+4*(i-1)):(8+4*(i-1))) = [compt1; compt2;
    compt3; compt4];
end
%-----------------------------------------------------------------%
```

```
determinante = det(matriz);

end
```

Routine 6 – Mass Depletion Dynamic Analysis for Rocket Applications.

```
%{
Dynamic Analysis Routine

- Gives beam data to Natural Frequency Routine.
- Changes mass distribution 'RhoA' and appends new frequencies to array.
- Plots the graphic with in-flight frequency variation.
Note: mass axis has to be adimentional and reversed (from 0 to 1, not from m_i>m_f
    to m_f).

by h4lfling.
%}

format compact;

p = 20/100;             % porcentagem de variação
varM = ones(1,n_seg);   % depleção uniforme de massa

%---------------------
% ex. depleção não-uniforme:
% varM = [.25 .5 .75 1]
%
% PROTÓTIPO PARA DEPLEÇÃO NÃO UNIFORME: (similar ao artigo)
% min_dep = .1;                          % depleção limite de 10% da máxima
% varM = [1:-(1-min_dep)/(n_seg-1):min_dep]; % variação linear da depleção
%---------------------

dRhoA = p*varM.*RhoA;   % variação de massa
mt = [];                % massa em função do tempo [kg]
freq_adim_t = [];       % freq. adimensionais em função do tempo

% %{
while ~any(RhoA<=1.1*dRhoA)
%{
      Condição: distribuição de massa não diminui a valores desprezíveis
      É necessário usar Md somente para analisar cenários de depleção não
      uniforme de massa, ex: [10 10 10] >> [8 9 9.5]
%}
    mt = [mt sum(RhoA.*L)]; % salva massa total
    try
        freq_dim = falsaposicao(modos,L0,R1,R2,T1,T2,EI,RhoA,Li,a);
        % [rad/s]
```

```
        freq_adim_t = [freq_adim_t;
            ( L0^4 * freq_dim.^2 * RhoA(1) / EI(1) ).^(1/4)]; % [-]
        %freq_cic = freq_dim/(2*pi);
        %   [Hz]
    catch
        freq_dim = zeros(1,modos);freq_adim_t = freq_dim;
        freq_cic = freq_dim;
    end

    RhoA = RhoA - dRhoA;    % recalcula as distribuições de massa
    %a = a; % forças axiais calculadas para RhoA ( em implementação )
end
% %}

%--------------------- DISPLAY MASS STATISTICS
% RhoA
% mt(1) % kg
% disp('kg')
% mt(length(mt))*1000 % g
% disp('g')
%---------------------

% %{
%--------------------- PLOTTING
plot_separate = 0;
% [1] for plotting modes separate
try
    % colunas de freq_adim_t mostram variação de cada freq. adim. no tempo
    if plot_separate == 0
        plot(mt,freq_adim_t,'LineWidth',1.5)   % i-ésima frequência em função da
    massa
        set(gca, 'xdir', 'reverse') % invertendo eixo das massas
        xlabel('Massa [kg]')
        ylabel('Frequência Adimensional [-]')
        title('Modos de vibração ao longo do voo')
    else
        for i=1:modos
            figure
            plot(mt,freq_adim_t(:,i),'LineWidth',1.5)
            set(gca, 'xdir', 'reverse')
            xlabel('Massa [kg]')
            ylabel('Frequência Adimensional [-]')
            title([num2str(i),'º Modo de vibração ao longo do voo'])
        end
    end
catch
    disp('Erro na plotagem.');
```

```
end
%----------------------
% %}

format loose;
```

APPENDIX B– FEM MATLAB ROUTINES

Routine 7 – Beam Element Setup.

```
clear, clc, close all
% MEF crazy beam by masthierryi - LAMEC  - UFPI
% with notes and holes dug by h4lfling.
tic()

% OUTPUT ------------------------------------------------------------------
BT = 1; % Beam theories: 1 = EBT; 2 = RBT; % 3 = SBT; 4 = TBT
modes = 1:4; % number of displayed frequencies and modes
% ------------------------------------------------------------------------

% INPUT -------------------------------------------------------------------

% Input type 2%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% ------------------------------------------------------------------------
inp = 1; % beam index, for each beam, add one on its index
% parameters  ¨ ¨ ¨
```

```
data(inp).geo = 1; % lt = data(inp).n_el;

% multilayer data ¨ ¨ ¨
```

```
% ------------------------------------------------------------------------
FEA = calculations(data,BT,modes);

% basic analysis
calculations.Draw(FEA); %2.1s for 100 el
ShapeModes(FEA,BT,modes); % Mode shape for the chosen theory
% TheoriesModeShape(FEA,modes); % Theories shape modes comparing

% parameter analysis
type = 3;
% type = the desired relationship to plot
% type = 1: Euler-Bernoulli and Rayleight
% type = 2: Euler-Bernoulli and Shear
% type = 3: Euler-Bernoulli and Timoshenko
% Rho_natFreq(FEA,modes,data,type);
% E_natFreq(FEA,modes,data,type);
% slendernessR_natFreq(FEA,modes,data,type);

% h4lfling Notes: {Rho_natFreq,E_natFreq,slendernessR_natFreq} not working.

% ======================= PERIODICITY ANALYSIS ========================== %
% adapted by h4lfling.
% Frequency Spectra plot ------------------------
% Calculate prismatic beam frequencies for comparison
prismatic_data.segments = [
    7850, 205e9, 0.3, 0.01, 0.0250, 2, 1.0, 100;
];

% Create prismatic data structure
prismatic_data.n_c = 1;
prismatic_data.L_c = sum(prismatic_data.segments(:,8));
prismatic_data.L = sum(prismatic_data.segments(:,7)) * prismatic_data.n_c;
prismatic_data.noty = 2; prismatic_data.geo = 1;

lt = sum(prismatic_data.segments(:,end)) * prismatic_data.n_c;
prismatic_data.BC = [1, 0, 1; lt+1, 0, 1];

% Create fresh prismatic FEA object
prismatic_FEA = calculations(prismatic_data, BT, modes);
% calculations.Draw(prismatic_FEA);
prismatic_freqs = prismatic_FEA.result.natfreq(1:22);

% Plot frequency spectra
Freq_spectra(FEA, BT, 22, prismatic_freqs);
% ------------------------------------------------
```

```
% Evaluation of Saturation Parameter Effect -----
% NOTE: Edit function inside <calculations.m> for <stepped|bimat> beam
%       Unfortunately, this has to be done everytime. A better solution
%       could not be coded in time.
Periodic_Saturation(FEA,7:11,data);  % user should manually set ylim
Periodic_Saturation(FEA,18:22,data); % user should manually set ylim
% ========================================================================= %

% -------------------------------------------------------------------------

toc()

% masthierryi notes:
% time chart ------------------------------------------------------------
% only computting the frequencies: 0.07s
% [calculations.Draw]: % 1.2075174s for 100 elements
% [ShapeModes]: % 0.382775s
% [TheoriesModeShape]: % 0.5978928s
% [Rho_natFreq]: 22.3442085s for nmr = 320 repetitions with 2 theories
% [E_natFreq]: 22.472827s for nmr = 320 repetitions with 2 theories
% [slendernessR_natFreq]: 106.303478s for nmr = 1600 repetitions
%
% testing pc: xeon e5 2640 v4 2.2~2.5GHz CPU; 16GB RAM; 250GB SSD nvme 3rd gen;
% radeon gx580 sp
% -------------------------------------------------------------------------

%
% improvements to be made: ----------------------------------------------
% * r, s, A and I parameters for freq calculation for higher
% theories: what is these parameters on beams that have no constant
% material and geomtric parameters? talvez seja realmente o primeiro
% parametro na viga, usado pra calcular a freq e tals, ja q a mudaÃ§a
% relacionada a não constancia desses parametros é refletida nas matrizes
% de massa e rigidez. mas pq a freq da diferente entre vigas normais e
% modificadas?
% * tirar o U_disp e U_rot da resposta.
% * validate Rho_natFreq, E_natFreq; review slendernessR_natFreq, is not
% touching the x_axis even with nmr = 1600: ask slepton.
% * o Draw deve ser relacionar o form tbm, pros hollow muda mto
% * implement Inertia moment and calculations for rectangular box cross
% sections, then will be possible to make rectangular beam, same for elipse
%
% -------------------------------------------------------------------------
```

Routine 8 – Object Oriented FEM Class.

```
classdef calculations
  % MEF crazy beam by masthierryi - LAMEC  - UFPI
```

```
% ----
% h4lfling Notes:
% - Timoshenko might not be validated.
  % - Multilayer is currently incorrect.
% - {Rho_natFreq,E_natFreq,slendernessR_natFreq} not working (check <menu.m> for
  details).
  % ----
  properties
  data                        % beam data (struct)for each beam
  mesh                        % FEM mesh (struct) for each beam
  Beam                        % beam parameters for each beam
  layer                       % multilayer data
  % structure                   % final structure
  coupled_mesh                % coupled mesh for plottings
  modes                       % number of shape modes to be ploted

  r                           % rotatory inertia parameter for the entire beam
  s                           % shear deformation parameter for the entire beam
  S                           % slenderness ratio = 1/r
  gama                        % \frac{E}{\kappa G}
  L                           % sum of lenght of all coupled beams

  matrices
  % .K                        % global FEM stiffiness
  % .M                        % global FEM mass matrix
  % .CM                       % coupled mass matrix for all beams
  % .CK                       % coupled stiffness matrices

  U_disp                      % displacement matrix for ploting
  U_rot                       % rotation matrix for plotin

  result
  % .eigenvalues              % adimensional wave number
  % .natfreq                  % omega_n
  % .natfreqHz                % omega_n in [Hz]
  % .freq                     % dimensional omega_n
  critical                    % critical parameters

  end
  % ============================== FEM ================================ %
                                                                    methods
  %% CONSTRUCTOR
  function self = calculations(data,BT,modes)
      self.modes = modes;

      % Initialize self.data
      self.data = data;
```

```
    self.L = sum([data.L]); % sum of lenght of all coupled beams

    for input = 1:size(data,2)
        self.data = data;
        self.data(input) = data(input);

        self = FEM_setup(self,input);
        self = parameters(self,BT,input);
    end

    self = Eigenproblem(self); % setting the matrices
    self = Coupling(self); % coupling all beam matrices
    self = EigenSolve(self,BT); % solving for the frequencies
end

%% FEM SETTING
function self = FEM_setup(self,inp)

    % Geração automática da viga periódica
    self.data(inp).rho = []; self.data(inp).E   = []; self.data(inp).nu  = [];
self.data(inp).d   = [];
    segments = self.data(inp).segments;

    for i = 0:self.data(inp).n_c-1
        offset = i * self.data(inp).L_c; % Deslocamento baseado no número de
elementos
        elem_counter = 1 + offset;  % Começa sempre no primeiro elemento da cé
lula

        for j = 1:size(segments, 1)
            rho = segments(j, 1);
            E   = segments(j, 2);
            nu  = segments(j, 3);
            d1  = segments(j, 4);
            d2  = segments(j, 5);
            form = segments(j, 6);
            n_elements = segments(j, 8); % Número de elementos do segmento

            ini = elem_counter;         % Começo do segmento
            fin = elem_counter + n_elements - 1; % Fim do segmento

            % Adiciona os valores Ã s estruturas
            self.data(inp).rho{end+1,1} = [rho, ini, fin];
            self.data(inp).E{end+1,1}   = [E, ini, fin];
            self.data(inp).nu{end+1,1}  = [nu, ini, fin];
            self.data(inp).d{end+1,1}   = [d1, d2, form, ini, fin];
```

```
            % Atualiza o contador de elementos para o próximo segmento
            elem_counter = fin + 1;
        end
    end


    if self.data(inp).noty == 1 % common

        self.mesh(inp).n_el = self.mesh(inp).n_el; % number of elements
        self.mesh(inp).n_nodes = self.mesh(inp).n_el+1;  % number of nodes
        self.mesh(inp).l_el(1:self.mesh(inp).n_el) = ...
            (self.data(inp).L/self.mesh(inp).n_el)/2; % lenght of an isolated el

        % Coordinates increasing n_el times by n_el or L/n_el
        self.mesh(inp).coordinates = [linspace(0,self.data(inp).L,self.mesh(inp)
.n_nodes)]';

    elseif self.data(inp).noty == 2 % periodic

        % Total de elementos: soma dos elementos por cÃ©lula * nÃºmero de cÃ
©lulas unitÃ¡rias
        self.mesh(inp).n_el = sum(self.data(inp).segments(:,8)) * self.data(inp)
.n_c;
        self.mesh(inp).n_nodes = self.mesh(inp).n_el + 1;

        % Vetor de comprimentos dos elementos
        l_el_unit = [];

        for j = 1:size(self.data(inp).segments, 1)
            l_el_unit = [l_el_unit, repmat(
   ...
                (self.data(inp).segments(j, 7) / self.data(inp).segments(j, 8))
/2, ...
                1, self.data(inp).segments(j, 8))];
        end

        % Repete os comprimentos das cÃ©lulas unitÃ¡rias para preencher toda a
viga
        self.mesh(inp).l_el = [repmat(l_el_unit, 1, self.data(inp).n_c)]';

        % Coordinates increasing n_el times by n_el or L/n_el
        self.mesh(inp).coordinates = [0, cumsum([self.mesh(inp).l_el]')]';

    end
end
```

```
%% PARAMETERS #
function self = parameters(self,BT,input)

    self = self.geometry(self,input);
    self = self.material(self,input);

    % calculate each parameters for each element
    for el = 1:self.mesh(input).n_el
        E = self.Beam(input).E(el);
        A = self.Beam(input).A(el,1);
        I = self.Beam(input).I(el,1);
        k = self.Beam(input).k(el,1);
        G = self.Beam(input).G(el);
        l_e = self.mesh(input).l_el;
        % ------------------------------------------------------------

        % r and s parameters
        % ------------------------------------------------------------
        if BT == 1 % Euler BT ¨ ¨ ¨
```

```
        % ¨  ¨  ¨  
```

```
        % Stiffness global matrix pre-allocation
        GK = zeros(2*self.mesh(inp).n_nodes, 2*self.mesh(inp).n_nodes);
        % Mass global matrix
        GM = GK;
        % aux for allocating the elementar matrices on global
        satus = 1;

        for i = 1:self.mesh(inp).n_el % Loop for each element
        L_e = self.mesh(inp).l_el(i);
        % ==============================================================

            % Shape functions
            % --------------------------------------------------------
            s_e = self.Beam(inp).s_e(i);
            fc = 1/(4*(3*s_e^2+1)); % Constant for the function
            for j = 1:length(x)

                % functions for deflection ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨
¨ ¨

                Nd(1,j) = (2*(3*s_e^2+1)-3*(2*s_e^2+1)*x(j)+1*x(j)^3);
                Nd(2,j) = L_e*((3*s_e^2+1)-1*x(j)-(3*s_e^2+1)*x(j)^2+1*x(j)^3);
                Nd(3,j) = (2*(3*s_e^2+1)+3*(2*s_e^2+1)*x(j)-1*x(j)^3);
                Nd(4,j) = L_e*(-(3*s_e^2+1)-1*x(j)+(3*s_e^2+1)*x(j)^2+1*x(j)^3);
                Nd(:,j) = Nd(:,j)*fc;
                % functions for bending ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨
¨ ¨

                Ns(1,j) = (3*x(j)^2-3)/L_e;
                Ns(2,j) = (-1-2*(3*s_e^2+1)*x(j)+6*s_e^2+3*x(j)^2);
                Ns(3,j) = (3-3*x(j)^2)/L_e;
                Ns(4,j) = (-1+2*(3*s_e^2+1)*x(j)+6*s_e^2+3*x(j)^2);
                Ns(:,j) = Ns(:,j)*fc;
                % first derivate of Nd ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨
¨ ¨

                Nd_d(1,j) = (-3*(2*s_e^2+1)+3*x(j)^2);
                Nd_d(2,j) = L_e*(-1-2*(3*s_e^2+1)*x(j)+3*x(j)^2);
                Nd_d(3,j) = (3*(2*s_e^2+1)-3*x(j)^2);
                Nd_d(4,j) = L_e*(-1+2*(3*s_e^2+1)*x(j)+3*x(j)^2);
                Nd_d(:,j) = Nd_d(:,j)*fc;
                % first derivate of Ns¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨
¨ ¨ ¨

                Ns_d(1,j) = (6*x(j))/L_e;
                Ns_d(2,j) = (-2*(3*s_e^2+1)+6*x(j));
                Ns_d(3,j) = (-6*x(j))/L_e;
                Ns_d(4,j) = (2*(3*s_e^2+1)+6*x(j));
                Ns_d(:,j) = Ns_d(:,j)*fc;
                % ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨ ¨
¨ ¨ ¨ ¨ ¨ ¨
```

```
            end
            % ---------------------------------------------------------

            % Final Beam ----------------------------------------
            rho = self.Beam(inp).rho(i); A = self.Beam(inp).A(i); E = self.Beam(
inp).E(i);
            I = self.Beam(inp).I(i); k = self.Beam(inp).k(i); G = self.Beam(inp)
.G(i);
            r_e = self.Beam(inp).r_e(i);

            if self.data(inp).geo == 1  % if normal geometry
                % constant for translation mass matrix
                C_tra = (rho).* (A).* L_e;
                % constant for rotation mass matrix
                C_rot = (r_e.^2).* (rho).* (A).* L_e^3;
                % constant for stiffness bending matrix (\frac{EI}{A})
                C_ben = ((E).* (I))./ L_e;
                % constant for stiffness shear matrix
                C_she = ((s_e.^2).* (k.* G.* (A).^2* L_e^3))./((E).* (I));

            else
                % constant for translation mass matrix
                C_tra = ((rho.*A) + (self.layer(inp).rho.* self.layer(inp).A)).*
 L_e;
                % constant for rotation mass matrix
                C_rot = ((r_e).^2).* (rho + (sum(self.layer(inp).rho, 2))).* (A
+ sum(self.layer(inp).A, 2)).* L_e^3;
                % constant for stiffness bending matrix (\frac{EI}{A})
                C_ben = ( ( E.* I ) + (self.layer(inp).E.* self.layer(inp).I) )
./ L_e; %
                % constant for stiffness shear matrix
                C_she = (s_e^2.* (k.* G.* (A + sum(self.layer(inp).A, 2))).^2*
L_e^3)./( (E + sum(self.layer(inp).E, 2)).* (I + sum(self.layer(inp).I, 2)));
            end
            % ---------------------------------------------------------

            % Computing the element matrix
            % ---------------------------------------------------------
            M_tra = Nd * diag(w) * Nd';
            M_rot = Ns * diag(w) * Ns';
            K_ben = Ns_d * diag(w) * Ns_d';
            K_she = (Nd_d/self.mesh(inp).l_el(i)-Ns) * diag(w) * (Nd_d/self.mesh
(inp).l_el(i)-Ns)';

            % Complete mass element matrix
            M_m = M_tra.*C_tra(1) + M_rot.*C_rot(1);
            % Complete stiffness element matrix
```

```
            M_k = K_ben.*C_ben(1) + K_she.*C_she(1);
            % ----------------------------------------------------------

            % alocating the element matrices on the global
            % ----------------------------------------------------------
            finis = satus + 3;
            GM(satus:finis,satus:finis) = GM(satus:finis,satus:finis) + M_m(:,:)
;
            GK(satus:finis,satus:finis) = GK(satus:finis,satus:finis) + M_k(:,:)
;
            satus = satus + 2;
            % ----------------------------------------------------------

        end % end element loop
        % ==============================================================
        clear A C_ben C_rot C_she C_tra E G I K_ben K_she L_e M_k M_m M_rot
M_tra ...
        Nd Nd_d Ns Ns_d fc i j k r_e rho s_e BC_v % also reset BC_v

        % Boundary Conditions
        % --------------------------------------------------------------
        % auxiliary vector that transforms BC conditions into a column array
        BC_c = reshape([self.data(inp).BC(:, 2), self.data(inp).BC(:, 3)].', [],
 1);

        % defines the position of the dofs where the conditions are applied
        BC_pos = sort([2*self.data(inp).BC(:,1);2*self.data(inp).BC(:,1)-1]);

        % pre-allocation of vector with all dof boundary condition
        BC_v(1:2*self.mesh(inp).n_nodes,1) = 1;

        % adjust each dof condition
        for l = 1:size(BC_c)
            BC_v(BC_pos(l)) = BC_c(l);
        end

        % recreate the global matrices with only unconstrained dofs
        % ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨
  ¨   ¨   ¨   ¨   ¨   ¨   ¨
        GM = GM(logical(BC_v), :,:); % Remove rows
        GM = GM(:, logical(BC_v),:); % Remove columns
        GK = GK(logical(BC_v), :,:); % Remove rows
        GK = GK(:, logical(BC_v),:); % Remove columns
        self.mesh(inp).BC_v = BC_v; % vector with boundary conditions
        % ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨   ¨
  ¨   ¨   ¨   ¨   ¨   ¨   ¨
```

```
        self.matrices.M{inp} = GM;
        self.matrices.K{inp} = GK;
        % ------------------------------------------------------------

        % ID matrix
        % ------------------------------------------------------------
        self.mesh(inp).ID = ones(2, self.mesh(inp).n_nodes);

        % Apply boundary conditions directly using vectorized approach
        bc_indices = self.data(inp).BC(:, 1);      % Nodes where BC applies
        bc_values = self.data(inp).BC(:, 2:3)';    % Corresponding BC values
        self.mesh(inp).ID(:, bc_indices) = bc_values;

        % Generate sequential IDs for free degrees of freedom
        free_dofs = (self.mesh(inp).ID == 1);      % Logical array for free DOFs
        self.mesh(inp).ID(free_dofs) = 1:nnz(free_dofs); % Assign sequential IDs
        % ------------------------------------------------------------


    end % end loop for different bems
    % ---------------------------------------------------------------

end % end the matrix

%% COUPLING ##
function self = Coupling(self)

    % get the size of matrices inside self.matrices.K
    N_sysn(:) = cellfun(@length, self.matrices.K);
    % pre-allocate the system global matrix
    self.matrices.CM = zeros(N_sysn(1) + sum(N_sysn(2:end)-2), N_sysn(1) + sum(
N_sysn(2:end)-2));
    % self.matrices.CM = zeros(N_sysn(1) + sum(N_sysn(2:end)), N_sysn(1) + sum(
N_sysn(2:end)));
    self.matrices.CK = self.matrices.CM;

    satus = 1; % aux for loccating the local matrices on the global
    for inp = 1:length(self.matrices.K)
        finis = satus + N_sysn(inp)-1; % same as satus

        % couples the mass matrices
        self.matrices.CM(satus:finis,satus:finis) = ...
            self.matrices.CM(satus:finis,satus:finis) + self.matrices.M{inp
}(:,:);

        % couples the stifffness matrices
        self.matrices.CK(satus:finis,satus:finis) = ...
```

```
            self.matrices.CK(satus:finis,satus:finis) + self.matrices.K{inp
}(:,:);

        satus = finis - 1;
    end
    clear satus finis inp N_sysn


    % coupling the mesh data for plotting the mode shapes
    % ----------------------------------------------------------------
    self.coupled_mesh.n_el = sum([self.mesh(:).n_el]); % number of elements
    self.coupled_mesh.n_nodes = self.coupled_mesh.n_el + 1;  % number of nodes
    self.coupled_mesh.l_el = (self.L/self.coupled_mesh.n_el)/2; % lenght of an
isolated el

    % coordinates
    self.coupled_mesh.coordinates = zeros(self.coupled_mesh.n_nodes, 1);

    % Flatten coordinates and apply offsets
    offset = 0; % Initialize offset
    current_index = 1; % Initialize insertion index

    for inp = 1:length(self.mesh)
        coords = self.mesh(inp).coordinates; % Current mesh coordinates
        start_idx = 1 + (inp > 1); % Skip the first point if not the first mesh
        n_points = numel(coords) - (inp > 1); % Adjust number of points
accordingly
        self.coupled_mesh.coordinates(current_index:current_index + n_points -
1) = coords(start_idx:end) + offset; % Adjust and add
        offset = offset + coords(end); % Update offset with the last point of
current mesh
        current_index = current_index + n_points; % Update insertion index
    end
    % ----------------------------------------------------------------

    % Coupled BC
    % ---------------------------------------------------------------
    % preallocate BC_c as a cell array
    BC_c = cell(length(self.data), 1);

    % fill BC_beam with boundary conditions
    for i = 1:length(self.data)
        if i == 1
            % first beam
            BC_c{i} = self.data(i).BC; % Ãšltimo nÃ³ livre
        else
            % other beams
```

```
            BC_c{i} = [self.data(i).BC(2:end, 1) + self.data(i-1).BC(2:end, 1)
-1,...
                    self.data(i).BC(2:end, 2:3)]; % Ignorar o primeiro nÃ³
        end
    end

    % combine all boundary conditions into the global BC matrix
    self.coupled_mesh.BC = vertcat(BC_c{:});
    % ----------------------------------------------------------------

    % Coupled ID matrix
    % ----------------------------------------------------------------
    self.coupled_mesh.ID = ones(2, self.coupled_mesh.n_nodes);

    % Apply boundary conditions directly using vectorized approach
    bc_indices = self.coupled_mesh.BC(:, 1);       % Nodes where BC applies
    bc_values = self.coupled_mesh.BC(:, 2:3)';    % Corresponding BC values
    self.coupled_mesh.ID(:, bc_indices) = bc_values;

    % Generate sequential IDs for free degrees of freedom
    free_dofs = (self.coupled_mesh.ID == 1);      % Logical array for free DOFs
    self.coupled_mesh.ID(free_dofs) = 1:nnz(free_dofs); % Assign sequential IDs
    % ----------------------------------------------------------------

    % Coupled boundary conditions vector
    % ----------------------------------------------------------------
    BC_c = reshape([self.coupled_mesh.BC(:, 2), self.coupled_mesh.BC(:, 3)].',
[], 1);

    % defines the position of the dofs where the conditions are applied
    BC_pos = sort([2*self.coupled_mesh.BC(:,1);2*self.coupled_mesh.BC(:,1)-1]);

    % pre-allocation of vector with all dof boundary condition
    self.coupled_mesh.BC_v(1:2*self.coupled_mesh.n_nodes,1) = 1;

    % adjust each dof condition
    for l = 1:size(BC_c)
        self.coupled_mesh.BC_v(BC_pos(l)) = BC_c(l);
    end
    % ----------------------------------------------------------------

end

%% EIGENSOLVING

function self = EigenSolve(self,BT)
```

```
    r_t = self.r(1); s_t = self.s(1);
    A = self.Beam(1).A(1);
    I = self.Beam(1).I(1);
    k = self.Beam(1).k(1);
    E = self.Beam(1).E(1);
    rho = self.Beam(1).rho(1);
    G = self.Beam(1).G(1);

    % Computing the eigenvectors and eigenvalues
    [a_vet,a_val] = eig(self.matrices.CM\self.matrices.CK);

    % Natural frequencies
    omega = sqrt(diag(a_val));

    % Arranging the frequencies in ascending order
    [self.result.natfreq,Ind] = sort(omega);

    % Natural frequencie (hz)
    self.result.natfreqHz = self.result.natfreq./(2*pi);

    % Critical parameters ---------------------------------------------
    % critical b eigenvalue
    self.critical.b = 1/(r_t*s_t);

    % Critical frequency
    self.critical.f = sqrt((k.*I.*A)./(rho.*I));

    % Critical wavenumber = Critical frequency [Hz]
    self.critical.omega = (1/(2*pi))*sqrt((k.*G.* A)./(rho.*I));
    % ------------------------------------------------------------

    % Solution eigenvalues and Dispersion relations ===================
    B = sqrt((rho* A* self.L^4)/ (E* I));
    b = self.result.natfreq*B;

    % Solution eigenvalues and Dispersion relations ===================
    if BT == 1 % Euler-Bernoulli -------------------------------------
         % Beta
        self.result.dispersion(:,1) = sqrt(b);
        self.result.freq(:,1) = B*(b./(2*pi));

    elseif BT == 2 % Rayleight Beam ----------------------------------
        % Beta
        self.result.dispersion(:,1) = sqrt((b.^2/2).*(r_t.^2 + sqrt(r_t.^4 + 4./
b.^2)));
        % Alpha
        self.result.dispersion(:,2) = sqrt((b.^2/2).*(-r_t.^2 + sqrt(r_t.^4 +
```

```
4./b.^2)));

        self.result.freq(:,1) = B*(sqrt(self.result.eigenvalues(:,1).^2 ...
            - self.result.dispersion(:,2).^2)/2*pi*r_t);

    elseif BT == 3 % Shear Beam ---------------------------------------
        % Beta
        self.result.dispersion(:,1) = sqrt((b.^2/2).*(s_t.^2 + sqrt(s_t.^4 + 4./
b.^2)));
        % Alpha
        self.result.dispersion(:,2) = sqrt((b.^2/2).*(-s_t.^2 + sqrt(s_t.^4+ 4./
b.^2)));

        self.result.freq(:,1) = B*(sqrt(self.result.eigenvalues(:,1).^2 ...
            - self.result.dispersion(:,2).^2)/2*pi*s_t);

    elseif BT == 4 % Timoshenko Beam ----------------------------------
        % Beta
        self.result.dispersion(:,1) = sqrt((b.^2/2).*((r_t.^2+s_t.^2) ...
            + sqrt((r_t.^2-s_t.^2)^2 + 4./b.^2)));
        % Alpha before critical frequency
        self.result.dispersion(:,2) = sqrt((b.^2/2).*(-(r_t.^2+s_t.^2) ...
            + sqrt((r_t.^2-s_t.^2)^2 + 4./b.^2)));
        % Alpha after the critical frequency
        self.result.dispersion(:,3) = sqrt((b.^2/2).*((r_t.^2+s_t.^2) ...
            - sqrt((r_t.^2-s_t.^2)^2 + 4./b.^2)));

        for j = 1:size(b)
            if b(j) < self.critical.b
                % Natural frequency before critical frequency
                self.result.freq(j,1) = B*(sqrt(self.result.dispersion(j,1).^2
...
                    - self.result.dispersion(j,2).^2)/ ...
                    (2*pi*sqrt(r_t.^2+s_t.^2)));
            elseif b(j) > self.critical.b
                % Natural frequency after critical frequency
                self.result.freq(j,2) = B*(sqrt(self.result.dispersion(j,1).^2
...
                    + self.result.dispersion(j,3).^2)/ ...
                    (2*pi*sqrt(r_t.^2+s_t.^2)));
            end
        end

    end % ==============================================================

    % %Dispersion relationship (wave number) ============================
    %
```

```
    % if BT == 1 %Euler-Bernoulli ----------------------------------------
    %     self.result.eigenvalues = sqrt(self.result.natfreq.*self.L.*sqrt(rho/
E)*self.S(1));
    %
    % elseif BT == 2 %Rayleight Beam -------------------------------------
    %     % Alpha
    %     self.result.eigenvalues(:,1) = (b./sqrt(2)).*sqrt(-(1/self.S(1))^2 +
sqrt((1/self.S(1))^4 + 4./b.^2));
    %     % Beta
    %     self.result.eigenvalues(:,2) = (b./sqrt(2)).*sqrt((1/self.S(1))^2 +
sqrt((1/self.S(1))^4 + 4./b.^2));
    %
    % elseif BT == 3 %Shear Beam ----------------------------------------
    %     % Alpha
    %     self.result.eigenvalues(:,1) = (b./sqrt(2)).*sqrt(-(1/self.S(1)*self.
gama(1))^2+...
    %         sqrt((1/self.S(1)*self.gama(1))^4 + 4./b.^2));
    %     % Beta
    %     self.result.eigenvalues(:,2) = (b./sqrt(2)).*sqrt((1/self.S(1)*self.
gama(1))^2+...
    %         sqrt((1/self.S(1)*self.gama(1))^4 + 4./b.^2));
    %
    % elseif BT == 4 %Timoshenko Beam ------------------------------------
    %     % Alpha before critical frequencie
    %     self.result.eigenvalues(:,3) = (b./sqrt(2)).*sqrt(((1/self.S(1))^2
+...
    %         ((1/self.S(1))*self.gama(1))^2) - sqrt( ((1/self.S(1))^2 -...
    %         ((1/self.S(1))*self.gama(1))^2)^2 + 4./b.^2));
    %
    %     % Alpha after the critical frequencie
    %     self.result.eigenvalues(:,2) = (b./sqrt(2)).*sqrt(-((1/self.S(1))^2
+...
    %         ((1/self.S(1))*self.gama(1))^2) + sqrt( ((1/self.S(1))^2 -...
    %         ((1/self.S(1))*self.gama(1))^2)^2 + 4./b.^2));
    %
    %     % Beta
    %     self.result.eigenvalues(:,1) = (b./sqrt(2)).*sqrt(((1/self.S(1))^2
+...
    %         ((1/self.S(1))*self.gama(1))^ 2) + sqrt( ((1/self.S(1))^2 -...
    %         ((1/self.S(1))*self.gama(1))^2)^2 + 4./b.^2));
    %
    % end % ==============================================================

    % rigid body movement
    % Rbm = length(self.result.natfreq(self.result.natfreq<8));
    %
    % % FREQ DISPLAY
```

```
% frequencies(1,:) = self.result.eigenvalues(1+Rbm:self.modes+Rbm);
% display(frequencies)

% clear

% Pre-allocation ---------------------------------------------------
A_vet = zeros(length(Ind)); % eigenvectors ordened
% ------------------------------------------------------------------

% arranging the eigenvectors in ascending order
for i=1:length(Ind)
    A_vet(:,i)=a_vet(:,Ind(i));
end

% Adding the contribution of the loop constants
BC_all = vertcat(self.coupled_mesh.BC);
cont_disp = sum(1 - BC_all(:,2));
cont_rot = sum(1 - BC_all(:,3));

% Displacement -----------------------------------------------------
disp = nonzeros(self.coupled_mesh.ID(1,:));
self.U_disp = zeros(length(disp) + cont_disp,length(A_vet));
% Re-ordenate eigenvectors rows of displacement
for i=1:length(disp)
    self.U_disp(i+1-self.coupled_mesh.BC_v(1),:) = A_vet(disp(i),:);
end
% Displacement normalization
% self.U_disp = self.U_disp./max(abs(self.U_disp));%max() abs()
self.U_disp = normalize(self.U_disp,"norm");
for  j = 1:size(A_vet)
    if  self.U_disp(2,j) <= 0
        self.U_disp(:,j) = -self.U_disp(:,j);
    end
end
self.U_disp = self.U_disp.*(-1);
%for inverting the plot vertically. same on U_rot.
%-----------------------------------------------------------------

% Rotation ---------------------------------------------------------
rot = nonzeros(self.coupled_mesh.ID(2,:));
self.U_rot = zeros(length(rot) + cont_rot,length(A_vet));
% Re-ordenate eigenvectors rows of rotation
for i=1:length(rot)
    self.U_rot(i+1-self.coupled_mesh.BC_v(2),:) = A_vet(rot(i),:);
end
% Rotation normalization
self.U_rot = self.U_rot./max(max(abs(self.U_rot)));
```

```
    for  j = 1:size(A_vet)
        if  self.U_rot(2,j) <= 0
            self.U_rot(:,j) = -self.U_rot(:,j);
        end
    end
    % self.U_rot = self.U_rot.*(-1);
    %-----------------------------------------------------------------

end % end FrequenciesComp

                                                                    end
% ============================ ANALYSIS ============================ %
                                                                methods
%% MODE SHAPES
function self = ShapeModes(self,BT,modes)

    % Removing rigid body movement
    Rbm = length(self.result.natfreq(self.result.natfreq<8));

    modelTypes = {'EBT', 'RBT', 'SBT', 'TBT'};
    model = modelTypes{BT};

    maxdisp = max(max(self.U_disp(:,modes(1)+ Rbm:modes(end)+ + Rbm)))+0.05;
    maxrot = max(max(self.U_rot(:,modes(1)+ Rbm:modes(end)+ + Rbm)))+0.05;
    mindisp = min(min(self.U_disp(:,modes(1)+ Rbm:modes(end)+ + Rbm)))-0.05;
    minrot = min(min(self.U_rot(:,modes(1)+ Rbm:modes(end)+ + Rbm)))-0.05;

    % % Rotation mode shape ----------------------------------------
    % figure("Position",[460 40 500 300])
    % hold on
    % box on
    % grid on
    % title(sprintf('First %d rotation modes of a %s beam:',self.modes,model));
    % xlabel('\xi')
    % ylabel('V(\xi)')
    % axis([0 max(self.coupled_mesh.coordinates)/self.L minrot maxrot])
    % for  mode_cnt = modes(1):modes(end)
    %     plot(self.coupled_mesh.coordinates/self.L, self.U_rot(:,mode_cnt + Rbm
),...
    %         "DisplayName",sprintf('%dÃº mode',mode_cnt))
    %     legend('-DynamicLegend');
    % end
    % legend ('show');
    % legend('Location','southwest')
    % hold off
    % %-----------------------------------------------------------
    %
```

```
    %         % Normal mode shape ------------------------------------------
    % figure("Position",[460 342 500 300])
    % hold on
    % box on
    % grid on
    % title(sprintf('First %d mode of a %s beam:',self.modes,model));
    % xlabel('\xi')
    % ylabel('V(\xi)')
    % axis([0 max(self.coupled_mesh.coordinates)/self.L mindisp maxdisp])
    % for  mode_cnt = modes(1):modes(end)
    %     plot(self.coupled_mesh.coordinates/self.L,  self.U_disp(:,mode_cnt +
Rbm),  ...
    %         "DisplayName",sprintf('%dÂº mode',mode_cnt));
    %     legend('-DynamicLegend');
    % end
    % legend ('show');
    % legend('Location','southwest')
    % hold off
    % % ------------------------------------------------------------

    % HALF MODES ====================================================
    half  = 1:round(size(self.coupled_mesh.coordinates,1)/2);
    % Rotation mode shape ----------------------------------------
    figure("Position",[460 40 500 300])
    hold on
    box on
    grid on
    title(sprintf('Rotation modes of a %s beam:',model));
    xlabel('\xi')
    ylabel('V(\xi)')
    axis([0 max(self.coupled_mesh.coordinates(half))/self.L minrot maxrot])
    for  mode_cnt = modes(1):modes(end)
        plot(self.coupled_mesh.coordinates(half)/self.L, self.U_rot(half,
mode_cnt + Rbm),...
            "DisplayName",sprintf('%dÂº mode',mode_cnt))
        legend('-DynamicLegend');
    end
    legend ('show');
    legend('Location','southwest')
    hold off
    %-------------------------------------------------------------

            % Normal mode shape ------------------------------------------
    figure("Position",[460 342 500 300])
    hold on
    box on
    grid on
```

```
    title(sprintf('Shape modes of a %s beam:',model));
    xlabel('\xi')
    ylabel('V(\xi)')
    axis([0 max(self.coupled_mesh.coordinates(half))/self.L mindisp maxdisp])
    for  mode_cnt = modes(1):modes(end)
        plot(self.coupled_mesh.coordinates(half)/self.L,  self.U_disp(half,
mode_cnt + Rbm),  ...
            "DisplayName",sprintf('%dÃº mode',mode_cnt));
        legend('-DynamicLegend');
    end
    legend ('show');
    legend('Location','southwest')
    hold off
    % ------------------------------------------------------------

end % End of the plotting

%% ALL THEORIES SHAPE MODES COMPARING
function self = TheoriesModeShape(self,modes)
    nmodes = size(modes,2);

    % Removing rigid body movement
    Rbm = length(self.result.natfreq(self.result.natfreq<8));

    % Ploting all theories for this boundary condition ------------
    figure;
    box on
    grid on
    title(sprintf('%d Mode Shapes of all theories',modes));
    xlabel('\xi')
    ylabel('V(\xi)')
    for i = 1:4
        BT = i;

        for input = 1:size(self.data,2)
            self = parameters(self,BT,input);
        end

        self = Eigenproblem(self); % setting the matrices
        self = Coupling(self); % coupling all beam matrices
        self = EigenSolve(self,BT); % solving for the frequencies

        % Ploting the value for this theory
        for  mode_cnt = modes(1):modes(end)
            subplot(nmodes,1,mode_cnt);
            hold on
            p = plot(self.coupled_mesh.coordinates/self.L,self.U_disp(:,mode_cnt
```

```
 + Rbm));
            title(sprintf('mode %d',mode_cnt));
            axis([0 max(self.coupled_mesh.coordinates)/self.L -1.5 1.5])

            if i == 1
                p.LineStyle = '-';
                p.DisplayName = 'Euler';
            elseif i == 2
                p.LineStyle = '-.';
                p.DisplayName = 'Rayleigh';
            elseif i == 3
                p.LineStyle = ':';
                p.DisplayName = 'Shear';
            elseif i == 4
                p.LineStyle = '--';
                p.DisplayName = 'Timoshenko';
            end

        end

    end
    legend('Location','southoutside','Orientation','horizontal')
    hold off % end all theories

    % % ############
    % % EBT x TBT for various frequencies ---------------------------
    % figure;
    % box on
    % grid on
    %
    % xlabel('\xi')
    % ylabel('V(\xi)')
    % for i = [1,4]
    %     BT = i;
    %
    %     % Reruning the code for each beam theory
    %     self = parameters(self,BT);
    %     self = Eigenproblem(self);
    %
    %     % Ploting the value for this theory
    %     for  mode_cnt = 1:self.modes
    %         subplot(self.modes,1,mode_cnt);
    %         hold on
    %         p = plot(self.mesh.coordinates/self.data.L,self.U_disp(:,mode_cnt
 + RBM));
    %         title(sprintf('mode %d',mode_cnt));
    %         axis([0 max(self.mesh.coordinates)/self.data.L -1.5 1.5])
```

```
    %
    %         if i == 1
    %             p.LineStyle = '-';
    %             p.DisplayName = 'Euler';
    %         elseif i == 4
    %             p.LineStyle = '--';
    %             p.DisplayName = 'Timoshenko';
    %         end
    %
    %     end
    %
    % end
    % legend('Location','southoutside','Orientation','horizontal')
    % hold off % end RBT x TBT various freq rotation
    %
    % % ############
    % % EBT x TBT for various frequencies (Rotation) ----------------
    % figure;
    % box on
    % grid on
    % title(sprintf('First %d Mode Shapes of EBT and TBT theories (Rotation)',
self.modes));
    % xlabel('\xi')
    % ylabel('V(\xi)')
    % for i = [1,4]
    %     BT = i;
    %
    %     % Reruning the code for each beam theory
    %     self = parameters(self,BT);
    %     self = Eigenproblem(self);
    %
    %     % Ploting the value for this theory
    %     for  mode_cnt = 1:self.modes
    %         subplot(self.modes,1,mode_cnt);
    %         hold on
    %         p = plot(self.mesh.coordinates/self.data.L,self.U_rot(:,mode_cnt +
 RBM));
    %         title(sprintf('mode %d',mode_cnt));
    %         axis([0 max(self.mesh.coordinates)/self.data.L -1.5 1.5])
    %
    %         if i == 1
    %             p.LineStyle = '-';
    %             p.DisplayName = 'Euler';
    %         elseif i == 4
    %             p.LineStyle = '--';
    %             p.DisplayName = 'Timoshenko';
    %         end
```

```
    %
    %     end
    %
    % end
    % legend('Location','southoutside','Orientation','horizontal')
    % hold off % end RBT x TBT various freq rotation

end % end allthoeries function

%% Rho x natFreq #####
function self = Rho_natFreq(self,modes,data,type)
        nmodes = size(modes,2);

        % Suport variable
        second_spec = zeros(1,nmodes); % Is the change for alphas
        Rbm = length(self.result.natfreq(self.result.natfreq<8));

        % Number of increases necessary for 1/S goes from 0 to 0.4
        nmr = 320; %320
        % Prealocating the frequencies and changes matrix
        frequencies = ones(nmr,nmodes*5);

        % Reseting the diameter
        rhov = linspace (1000,12000,nmr);

        for j = 1:nmr
            BT = 1;

            % ---------------------------------------------------------
            for input = 1:size(data,2)
                self.data(input).rho{1}(1) = rhov(j);
                self = parameters(self,BT,input);
            end

            self = Eigenproblem(self); % setting the matrices
            self = Coupling(self); % coupling all beam matrices
            self = EigenSolve(self,BT); % solving for the frequencies

            wv_n = self.result.dispersion; % wave number
            natF_Hz = self.result.natfreqHz;
            % ---------------------------------------------------------

            frequencies(j,1:nmodes) = wv_n(modes(1)+Rbm : modes(end)+Rbm,1);

            frequencies(j,nmodes+1:2*nmodes) = natF_Hz(modes(1)+Rbm : modes(end)
+Rbm,1);
```

```
            % Calculating the frequencies for the next Beam theory ----
            BT = 1+type;

            % -------------------------------------------------------
            for input = 1:size(data,2)
                self.data(input).rho{1}(1) = rhov(j);
                self = parameters(self,BT,input);
            end

            self = Eigenproblem(self); % setting the matrices
            self = Coupling(self); % coupling all beam matrices
            self = EigenSolve(self,BT); % solving for the frequencies

            wv_n = self.result.dispersion; % wave number
            natF = self.result.natfreq;
            natF_Hz = self.result.natfreqHz;
            omega_crit = self.critical.omega;
            % -------------------------------------------------------

            A = self.Beam(1).A(1);
            I = self.Beam(1).I(1);
            E = self.Beam(1).E(1);
            rho = self.Beam(1).rho(1);

            % Alpha relation for Rayleight and shear beam, and Beta for
            % Timoshenko. Change made because Alpha of tbt change
            frequencies(j,2*nmodes+1:3*nmodes) = wv_n(modes(1)+Rbm : modes(end)+
Rbm,1);

            if type == 1
                % Beta relation
                frequencies(j,3*nmodes+1:4*nmodes) = wv_n(modes(1)+Rbm : modes(
end)+Rbm,2);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF_Hz(modes(1)+Rbm :
modes(end)+Rbm,1);

            elseif type == 2
                % Beta relation
                frequencies(j,3*nmodes+1:4*nmodes) = wv_n(modes(1)+Rbm : modes(
end)+Rbm,2);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF_Hz(modes(1)+Rbm :
modes(end)+Rbm,1);
```

```
            elseif type == 3
                B = sqrt(rho*A/(E*I))*natF*self.L^2;

                for i = 1:nmodes
                    if B(i+Rbm) > ((self.S(1))^2/self.gama(1))
                        second_spec(i) = 1;
                    end
                    % Alpha relation
                    frequencies(j,3*nmodes+i) = wv_n(i+Rbm,2+second_spec(i));
                end

                % Critical frequency
                frequencies(j,5*nmodes+1) = omega_crit(1);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF_Hz(modes(1)+Rbm :
modes(end)+Rbm,1);

            end

        end

        % Ploting the frequencies -------------------------------------
        figure
        hold on
        box on
        if type == 1
            el2 = plot(rhov(:),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(rhov(:),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Rayleight','color','b','HandleVisibility', 'off')
;
        elseif type == 2
            el2 = plot(rhov(:),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(rhov(:),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Shear','color','b','HandleVisibility', 'off');
        elseif type == 3
            el2 = plot(rhov(:),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(rhov(:),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Timoshenko','color','b','HandleVisibility', 'off'
);
        end
```

```
        % Legend ploting
        el2(1).HandleVisibility = 'on';
        sl(1).HandleVisibility = 'on';

        xlabel('Rho (Kg/m^3))')
        ylabel('\omega (Hz)')
        axis('tight')
        title('Natural Frequency x Specific Weight')
        legend('show','Location','northeast')

        hold off
        % ------------------------------------------------------------
end
%% E x natFreq #####
function self = E_natFreq(self,modes,data,type)
        nmodes = size(modes,2);

        % Suport variable
        second_spec = zeros(1,nmodes); % Is the change for alphas
        Rbm = length(self.result.natfreq(self.result.natfreq<8));

        nmr = 320; % number of repetitions
        % Prealocating the frequencies and changes matrix
        frequencies = ones(nmr,nmodes*5);

        % Reseting the diameter
        youngv = linspace (110e9,207e9,nmr);

        for j = 1:nmr
            BT = 1;

            % ---------------------------------------------------------
            for input = 1:size(data,2)
                self.data(input).E{1}(1) = youngv(j);
                self = parameters(self,BT,input);
            end

            self = Eigenproblem(self); % setting the matrices
            self = Coupling(self); % coupling all beam matrices
            self = EigenSolve(self,BT); % solving for the frequencies

            wv_n = self.result.dispersion; % wave number
            natF_Hz = self.result.natfreqHz;
            % ---------------------------------------------------------

            frequencies(j,1:nmodes) = wv_n(modes(1)+Rbm : modes(end)+Rbm,1);
```

```
            % Relation for frequency curves
            if type == 1 || type == 3

                frequencies(j,nmodes+1:2*nmodes) = natF_Hz(modes(1)+Rbm : modes(
end)+Rbm,1);

            elseif type == 2

                frequencies(j,nmodes+1:2*nmodes) = natF_Hz(modes(1)+Rbm : modes(
end)+Rbm,1);

            end

            % Calculating the frequencies for the next Beam theory ----
            BT = 1+type;

           % -------------------------------------------------------
            for input = 1:size(data,2)
                self.data(input).E{1}(1) = youngv(j);
                self = parameters(self,BT,input);
            end

            self = Eigenproblem(self); % setting the matrices
            self = Coupling(self); % coupling all beam matrices
            self = EigenSolve(self,BT); % solving for the frequencies

            wv_n = self.result.dispersion; % wave number
            natF = self.result.natfreq; %#########
            natF_Hz = self.result.natfreqHz;
            omega_crit = self.critical.omega;

            A = self.Beam(1).A(1);
            I = self.Beam(1).I(1);
            E = self.Beam(1).E(1);
            rho = self.Beam(1).rho(1);
            % -------------------------------------------------------

            % Alpha relation for Rayleight and shear beam, and Beta for
            % Timoshenko. Change made because Alpha of tbt change
            frequencies(j,2*nmodes+1:3*nmodes) = wv_n(modes(1)+Rbm : modes(end)+
Rbm,1);

            if type == 1
                % Beta relation
                frequencies(j,3*nmodes+1:4*nmodes) = wv_n(modes(1)+Rbm : modes(
end)+Rbm,2);
```

```
                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF_Hz(modes(1)+Rbm :
modes(end)+Rbm,1);

            elseif type == 2
                % Beta relation
                frequencies(j,3*nmodes+1:4*nmodes) = wv_n(modes(1)+Rbm : modes(
end)+Rbm,2);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF_Hz(modes(1)+Rbm :
modes(end)+Rbm,1);

            elseif type == 3
                B = sqrt(rho*A/(E*I))*natF*self.L^2;

                for i = 1:nmodes
                    if B(i+Rbm) > ((self.S(1))^2/self.gama(1))
                        second_spec(i) = 1;
                    end
                    % Alpha relation
                    frequencies(j,3*nmodes+i) = wv_n(i+Rbm,2+second_spec(i));
                end

                % Critical frequency
                frequencies(j,5*nmodes+1) = omega_crit(1);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF_Hz(modes(1)+Rbm :
modes(end)+Rbm,1);

            end
        end

        % Ploting the frequencies -------------------------------------
        figure
        hold on
        box on
        if type == 1
            el2 = plot(youngv(:),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(youngv(:),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Rayleight','color','b','HandleVisibility', 'off')
;
        elseif type == 2
```

```
            el2 = plot(youngv(:),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(youngv(:),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Shear','color','b','HandleVisibility', 'off');
        elseif type == 3
            el2 = plot(youngv(:),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(youngv(:),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Timoshenko','color','b','HandleVisibility', 'off'
);
        end

        % Legend ploting
        el2(1).HandleVisibility = 'on';
        sl(1).HandleVisibility = 'on';

        xlabel('Young`s modulus (GPa)')
        ylabel('\omega (Hz)')
        axis('tight')
        title('Natural Frequency x YoungÂ´s modulus')
        legend('show','Location','northeast')

        hold off
        % ------------------------------------------------------------
end
%% Slenderness Ratio x natFreq BY SLEPTON (2023) #####
function self = slendernessR_natFreq(self,modes,data,type)
        nmodes = size(modes,2);

        % Suport variable
        second_spec = zeros(1,nmodes); % Is the change for alphas
        Rbm = length(self.result.natfreq(self.result.natfreq<8));

        % Number of increases necessary for 1/S goes from 0 to 0.4
        nmr = 1600; %1600
        % Prealocating the frequencies and changes matrix
        frequencies = ones(nmr,nmodes*5);
        change = ones(nmr,3);

        % Reseting the diameter
        d_change = zeros(length(self.data),2);

        for j = 1:nmr
            BT = 1;
```

```
            % Recalculating the beam with new diameter
            d_change = d_change(1,1)+0.001; %[m] diameter;

            % ---------------------------------------------------------
            for input = 1:size(data,2)
                % self.data(input).d{1} = [d_change, d_change, 1, ...
                %     self.data(input).d{1}(1,4), self.data(input).d{1}(1,5)];
                self.data(input).d{1}(1:2) = d_change;
                % will not work for self.data(input).d(1,3), form =
                % hollow geometries
                self = parameters(self,BT,input);
            end

            self = Eigenproblem(self); % setting the matrices
            self = Coupling(self); % coupling all beam matrices
            self = EigenSolve(self,BT); % solving for the frequencies

            wv_n = self.result.dispersion; % wave number
            natF = self.result.natfreq;

            A = self.Beam(1).A(1);
            I = self.Beam(1).I(1);
            E = self.Beam(1).E(1);
            rho = self.Beam(1).rho(1);
            % ---------------------------------------------------------

            frequencies(j,1:nmodes) = wv_n(modes(1)+Rbm : modes(end)+Rbm,1);
            % Relation for frequency curves
            if type == 1 || type == 3

                frequencies(j,nmodes+1:2*nmodes) = natF(modes(1)+Rbm : modes(end
)+Rbm,1)*...
                    (1/self.S(1))*sqrt(rho*A*self.L^4/(E*I));

            elseif type == 2

                frequencies(j,nmodes+1:2*nmodes) = natF(modes(1)+Rbm : modes(end
)+Rbm,1)*...
                    self.gama(1)*(1/self.S(1))*sqrt(rho*A*self.L^4/(E*I));

            end

            % Calculating the frequencies for the next Beam theory
            BT = 1+type;

            % ---------------------------------------------------------
            for input = 1:size(data,2)
```

```
                % self.data(input).d{1} = [d_change, d_change, 1, ...
                %     self.data(input).d{1}(1,4), self.data(input).d{1}(1,5)];
                self.data(input).d{1}(1:2) = d_change;
                self = parameters(self,BT,input);
            end

            self = Eigenproblem(self); % setting the matrices
            self = Coupling(self); % coupling all beam matrices
            self = EigenSolve(self,BT); % solving for the frequencies

            wv_n = self.result.dispersion; % wave number
            natF = self.result.natfreq;
            % natF_Hz = self.result.natfreqHz;
            omega_crit = self.critical.omega;


            A = self.Beam(1).A(1);
            I = self.Beam(1).I(1);
            E = self.Beam(1).E(1);
            rho = self.Beam(1).rho(1);
            % ---------------------------------------------------------

            % Alpha relation for Rayleight and shear beam, and Beta for
            % Timoshenko. Change made because Alpha of tbt change
            frequencies(j,2*nmodes+1:3*nmodes) = wv_n(modes(1)+Rbm : modes(end)+
Rbm,1);

            if type == 1
                % Beta relation
                frequencies(j,3*nmodes+1:4*nmodes) = wv_n(modes(1)+Rbm : modes(
end)+Rbm,2);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF(modes(1)+Rbm : modes(
end)+Rbm,1)*...
                    (1/self.S(1))*sqrt(rho*A*self.L^4/(E*I));

            elseif type == 2
                % Beta relation
                frequencies(j,3*nmodes+1:4*nmodes) = wv_n(modes(1)+Rbm : modes(
end)+Rbm,2);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF(modes(1)+Rbm : modes(
end)+Rbm,1)*...
                    self.gama(1)*(1/self.S(1))*sqrt(rho*A*self.L^4/(E*I));
```

```
            elseif type == 3
                B = sqrt(rho*A/(E*I))*natF*self.L^2;

                for i = 1:nmodes
                    if B(i+Rbm) > ((self.S(1))^2/self.gama(1))
                        second_spec(i) = 1;
                    end
                    % Alpha relation
                    frequencies(j,3*nmodes+i) = wv_n(i+Rbm,2+second_spec(i));
                end

                % Critical frequency
                frequencies(j,5*nmodes+1) = omega_crit(1);

                % Relation for frequency curves
                frequencies(j,4*nmodes+1:5*nmodes) = natF(modes(1)+Rbm : modes(
end)+Rbm,1)*...
                    (1/self.S(1))*sqrt(rho*A*self.L^4/(E*I));
            end

            % X-axis
            change(j,1) = 1/self.S(1);
            change(j,2) = self.gama(1)/self.S(1);

        end

        % wave number curves ------------------------------------------
        figure
        hold on
        box on
        el = plot(change(:,1),frequencies(:,1:nmodes),'-',...
            'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off');
        if type == 1
            al = plot(change(:,1),frequencies(:,2*nmodes+1:3*nmodes),'--',...
                'DisplayName','\alpha','color','#A2142F','HandleVisibility', '
off');
            bl = plot(change(:,1),frequencies(:,3*nmodes+1:4*nmodes),'-.',...
                'DisplayName','\beta','color','b','HandleVisibility', 'off');
            xlabel('1/S')
            ylabel('Wave Number')
            axis([0 0.3 0 16])
        elseif type == 2
            al = plot(change(:,2),frequencies(:,2*nmodes+1:3*nmodes),'--',...
                'DisplayName','\alpha','color','#A2142F','HandleVisibility', '
off');
            bl = plot(change(:,2),frequencies(:,3*nmodes+1:4*nmodes),'-.',...
                'DisplayName','\beta','color','b','HandleVisibility', 'off');
```

```
            xlabel('\gamma/S')
            ylabel('Wave Number')
            axis([0 0.3 0 16])
        elseif type == 3
            ex = plot(change(:,1),frequencies(:,5*nmodes+1),'-',...
                'DisplayName','\beta_{crit}','color','#109411','HandleVisibility
', 'off');
            bl = plot(change(:,1),frequencies(:,2*nmodes+1:3*nmodes),'--',...
                'DisplayName','\beta','color','b','HandleVisibility', 'off');
            for i = 1:nmodes-1 % For all timoshenko alpha
                plot(change(:,1),frequencies(:,3*nmodes+i),'-.',...
                    'color','#A2142F','HandleVisibility', 'off'); % alpha
            end
            al = plot(change(:,1),frequencies(:,3*nmodes+nmodes),'-.',...
                'DisplayName','\alpha','color','#A2142F','HandleVisibility', '
off');
            xlabel('1/S')
            ylabel('Wave Number')
            axis([0 0.4 0 16])
        end

        % Legend ploting
        el(1).HandleVisibility = 'on';
        al(1).HandleVisibility = 'on';
        bl(1).HandleVisibility = 'on';
        ex(1).HandleVisibility = 'on';
        legend('show','Location','northeast')

        hold off
        % ------------------------------------------------------------

        % Ploting the frequencies curves ------------------------------

        figure
        hold on
        box on
        if type == 1
            el2 = plot(change(:,1),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(change(:,1),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Rayleight','color','b','HandleVisibility', 'off')
;
            xlabel('1/S')
            ylabel('\omega*1/S*(\rho*A*L^4/E*I)^{1/2}')
            axis([0 0.3 0 16])
        elseif type == 2
```

```
            el2 = plot(change(:,2),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(change(:,2),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Shear','color','b','HandleVisibility', 'off');
            xlabel('\gamma/S')
            ylabel('\omega*\gamma/S*(\rho*A*L^4/E*I)^{1/2}')
            axis([0 0.3 0 16])
        elseif type == 3
            el2 = plot(change(:,1),frequencies(:,nmodes+1:2*nmodes),'-',...
                'DisplayName','Euler','color','#EDB120','HandleVisibility', 'off
');
            sl =  plot(change(:,1),frequencies(:,4*nmodes+1:5*nmodes),'--',...
                'DisplayName','Timoshenko','color','b','HandleVisibility', 'off'
);
            xlabel('1/S')
            ylabel('\omega*1/S*(\rho*A*L^4/E*I)^{1/2}')
            % axis('tight')
            axis([0 0.4 0 6])
        end

        % Legend ploting
        el2(1).HandleVisibility = 'on';
        sl(1).HandleVisibility = 'on';
        legend('show','Location','southeast')

        hold off
end
%% natFreq x shape mode number + bandgap
% by masthierryi
% h4lfling dug several holes here
function self = Freq_spectra(self, BT, nmr, prismatic_freqs)
    modelTypes = {'Euler-Bernoulli', 'Rayleigh', 'Shear', 'Timoshenko'};
    model = modelTypes{BT};

    % Current periodic beam frequencies
    freqs = self.result.natfreq(1:nmr);
    maxfreq = freqs(end) + (freqs(end) - freqs(end-1));

    % Bandgap search algorithm (by masthierryi)
    % h4lfling notes:
    %     This approach is largely incorrect and WILL yield incorrect results.
    %     The user is encouraged to finetune the analysis on a case-by-case
    %     scenario, by changing the divisor on:
    %         if dfreq(i) < (dfreq(i-1)/2)
    %     The correct approach involves comparing frequency points against
    %     those of a reference prismatic beam.
```

```
    %     Then the relative difference should be calculated.
    %     masthierryi chose the easy way out on this one.
    % ----
    dfreq = diff(freqs);
    gap_indices = [];bandgap_starts = [];bandgap_ends = [];
    for i = 2:length(dfreq)
        if dfreq(i) < (dfreq(i-1)/2)
            gap_indices = [gap_indices, i-1];
            bandgap_starts = [bandgap_starts; freqs(i-1)];
            bandgap_ends = [bandgap_ends; freqs(i)];
        end
    end

    if ~isempty(bandgap_starts)
        gapvalue = bandgap_ends - bandgap_starts;
    else
        gapvalue = [];
    end

    % Plotting
    figure
    hold on
    box on
    % title(sprintf('Frequencies of %s Beam', model));
    xlabel('n')
    ylabel('\omega_n')
    axis([1 nmr 0 maxfreq])

    % Plot bandgaps first (so they appear behind the points)
    if ~isempty(gap_indices)
        % Create a dummy patch for legend (not displayed)
        bandgap_dummy = fill(NaN, NaN, 'blue', 'FaceAlpha', 0.1, 'EdgeColor', '
none', 'DisplayName', 'Bandgaps');

        for i = 1:length(bandgap_starts)
            % Plot each bandgap without legend entries
            fill([0, nmr+1, nmr+1, 0], [bandgap_starts(i), bandgap_starts(i),
bandgap_ends(i), bandgap_ends(i)], ...
                 'blue', 'FaceAlpha', 0.1, 'EdgeColor', 'none', '
HandleVisibility', 'off');

            % Add gap value text
            if ~isempty(gapvalue)
                str = {sprintf('%.2f', gapvalue(i))};
                text(2, (bandgap_starts(i) + bandgap_ends(i))/2, str, 'Color', '
black', ...
                     'HorizontalAlignment', 'left', 'FontWeight', 'bold');
```

```
            end
        end
    end

    % Plot periodic beam frequencies
    scatter(1:nmr, freqs, 25, 'blue', 'o', 'filled', 'MarkerEdgeColor', 'blue',
'LineWidth', 1.2);

    % Plot prismatic beam frequencies
    if nargin >= 4 && ~isempty(prismatic_freqs)
        scatter(1:min(nmr, length(prismatic_freqs)), prismatic_freqs(1:min(nmr,
length(prismatic_freqs))), ...
                25, 'red', 'o', 'MarkerEdgeColor', 'red', 'LineWidth', 1.2);
    end

    if nargin >= 4 && ~isempty(prismatic_freqs) && ~isempty(gap_indices)
        legend("Bandgaps", "Frequencies (Periodic)", "Frequencies (Prismatic)",
...
               "Location", "southeast")
    elseif nargin >= 4 && ~isempty(prismatic_freqs)
        legend("Frequencies (Periodic)", "Frequencies (Prismatic)", "Location",
"southeast")
    elseif ~isempty(gap_indices)
        legend("Bandgaps", "Frequencies (Periodic)", "Location", "southeast")
    else
        legend("Bandgaps", "Location", "southeast")
    end

    grid off
    hold off
end

%% natFreq x Periodic stepped saturation parameter
% h4lfling has been known to the function below
function self = Periodic_Saturation(self,modes,data)
    nmodes = size(modes,2);

    %{
        Domagalski, Ł. Comparison of the Natural Vibration Frequencies
        of Timoshenko and Bernoulli Periodic Beams.
        Materials 2021, 14, 7628. https://doi.org/10.3390/ma14247628
    %}

    % Reseting the saturation parameter
    alpha = 1/16:1/16:15/16; % h4lfling was here
    nmr = size(alpha,2);
```

```
    % Prealocating the frequencies and changes matrix
    frequencies = ones(nmr,nmodes);

    for j = 1:nmr
        BT = 1;

        % ---------------------------------------------------------
        for inp = 1:size(data,2)
            LS2 = (1-alpha(j))/2;
            LS1 = alpha(j);

            % DO COMMENT THE CONFIG NOT NEEDED BELOW!
            self.data(inp).segments = [
                % [rho, E, nu,        d1, d2, form,       section_length,
n_elements]

                7850, 205e9, 0.3,   0.01, 0.0250, 2,    LS2/10, 4; % simpa
stepped
                7850, 205e9, 0.3,   0.01, 0.0375, 2,    LS1/10, 4;
                7850, 205e9, 0.3,   0.01, 0.0250, 2,    LS2/10, 4;

                % 2700,  69e9, 0.3,   0.01, 0.0250, 2,    LS2/10, 4; %1 simpa
bimat
                % 7850, 205e9, 0.3,   0.01, 0.0250, 2,    LS1/10, 4;
                % 2700,  69e9, 0.3,   0.01, 0.0250, 2,    LS2/10, 4;
                ];
            % self.data(inp).segments = self.data.segments; % does not work :/
            self = FEM_setup(self,inp);
            self = parameters(self,BT,inp);
        end

        self = Eigenproblem(self); % setting the matrices
        self = Coupling(self); % coupling all beam matrices
        self = EigenSolve(self,BT); % solving for the frequencies

        wv_n = self.result.natfreq; % wave number
        % ---------------------------------------------------------

        frequencies(j,1:nmodes) = [wv_n(modes',1)]';
    end

    % Ploting the frequencies -------------------------------------
    figure
    hold on
    box on

    el2 = plot(alpha, frequencies, '-s', 'LineWidth', 1); % Plot lines
```

```
    % Reverse legend generation
    legends = arrayfun(@(k) sprintf('Mode %d', modes(k)), length(modes):-1:1, '
UniformOutput', false);

    legend(flip(el2), legends,'Location','northwest');

    xlabel('\alpha')
    ylabel('\omega')

    xticks(alpha)
    xticklabels(string(1:15));
    ax = gca;
    txt = text(1.1, 0, '$\times 1/16$', ...
        'Units', 'normalized', ...
        'HorizontalAlignment', 'right', ...
        'VerticalAlignment', 'bottom', ...
        'Interpreter', 'latex');

    axis('tight')

    % Comment out for final plot
    % title('Natural Frequency x Saturation parameter (\alpha)')

    % Use these to keep same scale on a bimat/stepped coupled plot
    % ylim([1.1e5 2.6e5]) % modes 18~22
    % ylim([1.5e4 6.4e4]) % modes 7~11

    hold off
    % ------------------------------------------------------------
end
                                                                end
% ============================ STATIC ============================== %
                                                    methods (Static)
%% LEGENDRE-GAUSS QUADRATURE BY GREG VON WINCKEL (2004)
function [x,w] = LGQ(~,N,a,b)

    % This script is for computing definite integrals using Legendre-Gauss
    % Quadrature. Computes the Legendre-Gauss nodes and weights  on an interval
    % [a,b] with truncation order N
    %
    % Suppose you have a continuous function f(x) which is defined on [a,b]
    % which you can evaluate at any x in [a,b]. Simply evaluate it at all of
    % the values contained in the x vector to obtain a vector f. Then compute
    % the definite integral using sum(f.*w);
    %
    % Written by Greg von Winckel in 02/25/2004
```

```
    N=N-1;
    N1=N+1; N2=N+2;
    xu=linspace(-1,1,N1)';
    % Initial guess
    y=cos((2*(0:N)'+1)*pi/(2*N+2))+(0.27/N1)*sin(pi*xu*N/N2);
    % Legendre-Gauss Vandermonde Matrix
    L=zeros(N1,N2);
    % Derivative of LGVM
    Lp=zeros(N1,N2);
    % Compute the zeros of the N+1 Legendre Polynomial
    % using the recursion relation and the Newton-Raphson method
    y0=2;
    % Iterate until new points are uniformly within epsilon of old points
    while max(abs(y-y0))>eps

        % Removed Lp for optimization, by slepton
        L(:,1)=1;
        L(:,2)=y;

        for k=2:N1
            L(:,k+1)=( (2*k-1)*y.*L(:,k)-(k-1)*L(:,k-1) )/k;
        end

        Lp=(N2)*( L(:,N1)-y.*L(:,N2) )./(1-y.^2);

        y0=y;
        y=y0-L(:,N2)./Lp;

    end
    % Linear map from[-1,1] to [a,b]
    x=(a*(1-y)+b*(1+y))/2;
    % Compute the weights
    w=(b-a)./((1-y.^2).*Lp.^2)*(N2/N1)^2;

end % end Legendre-Gauss Quadrature
%% MATERIAL
function self = material(self,input)

    % reorganizing values for each element ____________________________
    % depois fzer outr mÃ©todo sÃ³ pras seÃ§Ãµes bugadas e pra tapered, mudando
o
    % formato do input, deixar os tipos de input como comentario echamar os
    % metodos de geo em cada input, depois fzer o mesmo pra fgmb
    %
    %secoes bugadas colocar d1, d2..., dn
    %tapered: definir funÃ§Ã£o com o handle e fzer o calculo de cada nÃ³ no
```

```
metodo
    fields = {'E', 'rho'};
    for f = 1:numel(fields)
        masu = vertcat(self.data(input).(fields{f}){:});
        [values, intervals] = deal(masu(:, 1), masu(:, end-1:end));

        % calculate the size of the vector
        new_values = zeros(self.mesh(input).n_el, 1); % pre-allocate new_values

        % fill new_values with the values to its respective nodes
        for i = 1:numel(values)
            new_values(intervals(i,1) : intervals(i,2)) = values(i);
        end

        % store the vector on its due field
        self.Beam(input).(fields{f}) = new_values;
    end

    % Transversal Elasticity (G)
    self.Beam(input).G(:,1) = self.Beam(input).E./(2*(1 + self.Beam(input).nu));
    % self.Beam(input).G(:,1) = ones(1,size(self.Beam(input).E,1))*77.5e9;
    % ----------------------------------------------------------------
end
%% GEOMETRY
function self = geometry(self,input)

    n_el = self.mesh(input).n_el;
    fields = {'d1', 'd2', 'd3', 'nu'}; % d3 is the form
    new_values = zeros(n_el, numel(fields)); % PrÃ©-aloca matriz para todos os
campos

    masu_d = vertcat(self.data(input).d{:}); % Agrupa todos os d1, d2, d3
    masu_nu = vertcat(self.data(input).nu{:}); % Agrupa nu
    intervals = masu_d(:, end-1:end); % Assume que os intervalos sÃ£o os mesmos
para d e nu

    % Preenche os valores corretamente entre os nÃ³s iniciais e finais
    for i = 1:size(intervals, 1)
        idx_range = intervals(i,1):intervals(i,2); % Ã ndices do intervalo
        new_values(idx_range, 1:3) = new_values(idx_range, 1:3) + masu_d(i, 1:3)
; % d1, d2, d3
        new_values(idx_range, 4) = new_values(idx_range, 4) + masu_nu(i, 1); %
nu
    end

    % Armazena os resultados
    for f = 1:numel(fields)
```

```
        self.Beam(input).(fields{f}) = new_values(:, f);
    end

    clear fields f new_values intervals masu_d masu_nu idx_range;

    % ----------------------------------------------------------------
    for el = 1:self.mesh(input).n_el

        % rewritting the parameters for convenience
        d1 = self.Beam(input).d1(el);
        d2 = self.Beam(input).d2(el);
        nu = self.Beam(input).nu(el);

        if self.Beam(input).d3(el) == 1 % Circular¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨
¨

            A = pi/4*(d1.^2);
            I = pi/64*(d1.^4);
            k = 6*(1+nu)./(7+6*nu);

        elseif self.Beam(input).d3(el) == 2 % Rectangular ¨  ¨  ¨  ¨  ¨  ¨  ¨
            A = d1.*d2;
            I = (d1.*d2.^3)/12;
            k = 10*(1+nu)./(12+11.*nu);
            % k = 5/6;

        elseif self.Beam(input).d3(el) == 3 % Hollow Circle ¨  ¨  ¨  ¨  ¨  ¨
            A = pi*((d1/2)^2 - (d2/2)^2);
            I = (pi/2)*((d1/2)^4 - (d2/2)^4);
            k = (2 *(1+nu))/(4 + 3*nu);

        elseif self.Beam(input).d3(el) == 4 %Thin-Walled Square Tube¨  ¨
            A = d1^2 - d2^2;
            I = ((d1^4)/12) - ((d2^4)/12);
            k = (20*(1 + nu))/(48 + 39*nu);

        elseif self.Beam(input).d3(el) == 20 %Thin-Walled Square Tube¨  ¨
            A = 0.0097389;
            I = 0.0001171;
            k =  0.53066;

        end
        % ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨  ¨
¨  ¨  ¨  ¨  ¨  ¨  ¨
        % ----------------------------------------------------------------
                                        self.Beam(input).A(el,1) = A;
                                        self.Beam(input).I(el,1) = I;
                                        self.Beam(input).k(el,1) = k;
```

```
        end
        % ----------------------------------------------------------------

        % modifications ------------------------------------------------
        if self.data(input).geo ==  1
        else
            self = self.multilayer(self,input);
        end
        % ----------------------------------------------------------------
    end
    %% DRAW #####
    function self = Draw(self)

        x_ax = self.coupled_mesh.coordinates;

        if any([self.data.geo] == 2)
            d1 = [vertcat(self.Beam(:).d1)./2; self.Beam(end).d1(end)./2];
            d2 = d1;
        else
            d1 = [vertcat(self.Beam(:).d1)./2; self.Beam(end).d1(end)./2];
            d2 = [vertcat(self.Beam(:).d2)./2; self.Beam(end).d2(end)./2];
        end


        rho = [vertcat(self.Beam(:).rho); self.Beam(end).rho(end)];
        E = [vertcat(self.Beam(:).E); self.Beam(end).E(end)];
        nu = [vertcat(self.Beam(:).nu); self.Beam(end).nu(end)];


        scr_sz = get( groot, 'Screensize' );
        figure("Position",[0 0 500 scr_sz(end)])
        % ----------------------------------------------------------------
        subplot(3,1,1)
        hold on
        box on
        grid off
        % xlabel('L (m)')
        ylabel('heigth (m)')
        title('Side view of the beam and mesh')
        %
        % plot the vertical dimension line - - -
        stairs(x_ax,  [d1,-d1],"Color","k","LineWidth",1.5);

        % plot the nodes - - -
        scatter(x_ax,zeros(length(x_ax)),10,'MarkerEdgeColor',"k")

        % plot the central line - - -
```

```
    plot([0;max(x_ax)], [0;0],"Color","k","LineWidth",0.2)

    % % plot layers line - - -
    % stairs(x_ax,  [dl1,-dl1],"Color","b");

    axis([0 max(x_ax) -max(d1)-(max(d1)) max(d1)+max(d1)])
    legend ('off');
    hold off
    % ----------------------------------------------------------------
    % ----------------------------------------------------------------
    subplot(3,1,2)
    hold on
    box on
    grid off
    xlabel('L (m)')
    ylabel('width (m)')
    title('Top view of the beam and mesh')

    % plot the horizontal dimension line - - -
    stairs(x_ax,  [d2,-d2],"Color","k","LineWidth",1.7);

    % % plot layers line - - -
    % stairs(x_ax,  [dl1,-dl1],"Color","b");

    axis([0 max(x_ax) -max(d2)-(max(d2)) max(d2)+max(d2)])
    legend ('off');
    hold off
    % ----------------------------------------------------------------

    properties = {rho, E, nu};
    unities = {'kg/m^3', 'Pa', '~'};
    prop_name = {'\rho', 'E', '\nu'};
    colors =[1, 0.824, 0.408;  % Azul (Density)
             0.62, 0.886, 1;  % Vermelho (Young Modulus)
             0.706, 1, 0.322]; % Verde (Poisson Ratio)
    y_offsets = [0, -0.2, -0.4];
    % ----------------------------------------------------------------
    subplot(3,1,3)
    hold on
    box off
    grid off
    title('Material change from light to dark')
    % Material change colormap
    for i = 1:numel(properties)
        values = properties{i};
        normalized = 1 - (0.7 * (values - min(values)) / (max(values) - min(
values) + 1e-10));
```

```
        changes = [1; find(diff(values) ~= 0) + 1; length(values)]; % Indices of
 change

        % Plot segments
        for j = 1:length(changes)-1
            x_seg = x_ax(changes(j):changes(j+1));
            y_seg = y_offsets(i) * ones(size(x_seg));
            col = colors(i, :) *normalized(changes(j)); % Adjust color intensity
            plot(x_seg, y_seg, 'Color', col, 'LineWidth', 10);
            formatted_text = sprintf('%s [ %g, %g ] [ %s ]', prop_name{i}, min(
values), max(values), unities{i});
            text(0.01, y_offsets(i) + 0.1, formatted_text);
        end
    end
    axis([0 max(x_ax) (min(y_offsets)-0.2) (max(y_offsets)+0.2)])
    xticklabels([])
    yticklabels([])
    legend ('off');
    hold off
    % ----------------------------------------------------------------
    % end figure
end

%% MULTILAYER #####
% ---- Not validated ----
function self = multilayer(self,inp)

    % Inicializa os layers periÃ³dicos
    layers = self.data(inp).layer;
    n_layers = 1;
    n_celulas = self.data(inp).n_c;
    n_nodes = max(max(layers(:,5:6)));

    self.data(inp).layer_nodes = [];
    props = {'r', 'rho', 'E', 'nu'};
    for f = 1:4
        self.data(inp).(['layer_' props{f}]) = [];
    end

    % Preenche os dados das camadas periÃ³dicas
    for i = 0:n_celulas-1
        offset = i * n_nodes;
        self.data(inp).layer_nodes = [self.data(inp).layer_nodes; layers(:,5:6)
+ offset];
        for f = 1:4
            self.data(inp).(['layer_' props{f}]) = [self.data(inp).(['layer_'
props{f}]); layers(:, f)];
```

```
        end
    end

    for f = 1:4
        self.layer(inp).(props{f}) = zeros(max(max(self.data(inp).layer_nodes))
,1);
        for i = 1:size(self.data(inp).layer_r,1)
            ni = self.data(inp).layer_nodes(i, 1);
            nf = self.data(inp).layer_nodes(i, 2);
            self.layer(inp).(props{f})(ni:nf,1) = self.data(inp).(['layer_'
props{f}])(i,1);
        end
    end

    clear f new_values j ni nf;

    % rewritting ------------------------------------------------------
    form = self.Beam(inp).d3;
    db1 = self.Beam(inp).d1;
    db2 = self.Beam(inp).d2;
    r = self.layer(inp).r;
    % pre allocating
    self.layer(inp).dl1 = zeros(self.mesh(inp).n_el, size(r, 2));
    self.layer(inp).dl2 = zeros(self.mesh(inp).n_el, size(r, 2));
    % each layers dimensions
    for j = 1:size(r, 2)
        for i = 1:self.mesh(inp).n_el
            % Selecionar valor inicial ou acumulado de dl1
            if j == 1
                self.layer(inp).dl1(i, j) = (form(i) == 1 || form(i) == 2) * db1
(i) + ...
                    (form(i) == 3) * (db1(i) + db2(i)) + ...
                    (form(i) == 4) * db2(i);
            else
                self.layer(inp).dl1(i, j) = self.layer(inp).dl2(i, j-1);
            end

            % Calcular dl2 com base em dl1 e form(i)
            self.layer(inp).dl2(i, j) = self.layer(inp).dl1(i, j) + r(i, j) *
...
                (form(i) == 1 || form(i) == 3) + 2 * r(i, j) * (form(i) == 2 ||
form(i) == 4);
        end
    end
    % ----------------------------------------------------------------

    % ----------------------------------------------------------------
```

```
        for layer = 1:n_layers
        % layer = 1;
            for el = 1:self.mesh(inp).n_el

                % rewritting the parameters for convenience
                d2 = self.layer(inp).dl1(el,layer); % external
                d1 = self.layer(inp).dl2(el,layer); % internal
                nu = self.layer(inp).nu(el,layer);

                % ---------------------------------------------------------
                if form(el) == 1 || form(el) == 3 % pipe
                    A = pi*((d1/2)^2 - (d2/2)^2);
                    I = (pi/64)*((d1)^4 - (d2)^4);
                    k = (2 *(1+nu))/(4 + 3*nu);

                elseif form(el) == 2 || form(el) == 4 % box
                    A = d1^2 - d2^2;
                    I = ((d1^4)/12) - ((d2^4)/12);
                    k = (20*(1 + nu))/(48 + 39*nu);

                end
                % ---------------------------------------------------------
                                                self.layer(inp).A(el,layer) = A;
                                                self.layer(inp).I(el,layer) = I;
                                                self.layer(inp).k(el,layer) = k;
            end
        end
        % ------------------------------------------------------------------

    end % end multilayer

                                                                        end
                                                        methods (Access = private)
        % function result = cellcatcol(field, input, column)
        %     % only for convenience
        %     res = vertcat(self.data(input).(field){:});
        %     result = res(:,column);
        % end
                                                                        end

    % ============================== END ================================ %
end % classdef
```